

Stay in Your Lane:

Fast Arithmetic over Large Finite Rings with CKKS

Hyeonmin Choe , Robin Köstler, Tim Seuré 

University of Luxembourg, Esch-sur-Alzette, Luxembourg
hyeongmin.choe@uni.lu, robin.koestler@uni.lu, tim.seure@uni.lu

Preliminary draft — September 2026

Abstract. Recent CKKS-based constructions support homomorphic arithmetic modulo powers of two [GZ26] and more general moduli [LMWW26, WMGZ26]. We explore a complementary tradeoff: using polynomial rings native to CKKS simplifies arithmetic and refreshing but restricts the message spaces. Building on Kim’s interpretation of GBFV ciphertexts as CKKS ciphertexts [Kim25a], we use the decomposition $\mathbb{R}[X]/\langle X^N + 1 \rangle \cong (\mathbb{R}[X]/\langle X^n + 1 \rangle)^{N/n}$ [HHC19] into N/n polynomial lanes, where $n \in \llbracket 2, N \rrbracket$ is a power of two dividing the RLWE dimension N . Following GBFV [GV25], each lane represents a message in $\mathbb{Z}[X]/\langle X^n + 1, t \rangle$, where t is coprime to $X^n + 1$ over $\mathbb{Q}$. These message spaces include $\mathbb{Z}_{\beta^n+1}$ for $t = X - \beta$, products of modular integer rings, specific 128-, 256-, 512-, and 1024-bit prime fields, and extension fields such as $\mathbb{F}_{5^{16}}$, $\mathbb{F}_{17^4}$, and $\mathbb{F}_{37^{32}}$. Unlike GZ’s triangle encoding, our lanes occur at intermediate stages of conventional `CoeffToSlot` and `SlotToCoeff` transformations, avoiding additional encoding-specific linear maps. We thus “stay in our lane” by performing arithmetic and refreshing within CKKS-native polynomial rings. Our arithmetic bootstrapping restores ciphertext levels, bounds polynomial representatives, and reduces approximation error. At $N = 2^{16}$, it takes less than 4.5 seconds in every tested configuration, with little variation across t and n . Comparing our arithmetic modulo $2^n + 1$ with GZ’s arithmetic modulo 2^n , same-machine, same-library experiments show a $2.2\times$ – $3.8\times$ speedup for $n \in \llbracket 2^3, 2^6 \rrbracket$. Our bootstrapping time remains nearly constant as n grows, whereas GZ’s dense encoding-specific transformations increase its cost and, at larger n , exceed available memory, preventing direct comparisons. At $n = 2^{14}$, extrapolating GZ’s published timings suggests a $26\times$ speedup.

Keywords: Discrete CKKS · Encoding · Fully Homomorphic Encryption · Finite Rings

1 Introduction

We first review the development of large-modulus arithmetic using CKKS and position our contribution within this line of work. We then outline our construction in further detail.

1.1 General Overview

Homomorphic Encryption. Fully homomorphic encryption (FHE) enables a party to evaluate arbitrary circuits on encrypted inputs without learning their contents. Since Gentry’s first construction [Gen09], several families of schemes have been developed, with different approaches to representing and processing encrypted data. For instance, BGV and BFV [BGV12, Bra12, FV12] provide exact arithmetic over finite plaintext rings, whereas FHEW and TFHE [DM15, CGGI16] are particularly suited to Boolean computations. The CKKS scheme [CKKS17], in turn, was initially introduced for approximate arithmetic over real and complex numbers.

CKKS: From Approximate to Exact. The conventional plaintext space of CKKS is the quotient ring $\mathbb{R}[X]/\langle X^N + 1 \rangle$, where N is a power of two. Through the ring isomorphism $\mathbb{R}[X]/\langle X^N + 1 \rangle \cong \mathbb{C}^{N/2}$, a plaintext can be interpreted as a vector of $N/2$ complex numbers. Homomorphic addition and multiplication then act componentwise on these vectors, allowing many computations to be performed in parallel. Encoding and homomorphic evaluation introduce small errors, so the decrypted result approximates the desired computation. CKKS bootstrapping restores the capacity for further evaluation, allowing computations to continue beyond the depth supported by the initial ciphertext parameters. Since the first CKKS bootstrapping construction [CHK⁺18], its efficiency has been improved through faster linear transformations [HHC19], sparse-secret encapsulation [BTPH22], and the EvalRound approach [KPK⁺22]. More recent approaches, including SPRU [CK25], SHIP [CHKS25], and PaCo [CS26], exploit roots of unity to evaluate the decryption circuit homomorphically, while SCORE [Seu25] provides an optimized variant of SPRU for ciphertexts whose slots contain only real values.

Although approximation is central to this conventional interpretation, it does not prevent CKKS from supporting exact computations on discrete messages. If valid messages are sufficiently separated, a perturbed representation can still be decoded unambiguously. Moreover, knowledge of the discrete message space allows accumulated errors to be reduced homomorphically. This observation underlies the development of *discrete CKKS*, including the error-cleaning techniques of Drucker *et al.* [DMPS24], specialized bootstrapping for bits [BCKS24] and small integers [BKSS25], and constructions for finite-field arithmetic [SS26, KLN⁺26, JL26]. Functional bootstrapping further extends these capabilities by combining ciphertext refreshing with the evaluation of a prescribed function on a discrete domain, for example, [BKSS25, AKP25, AKPZ26]. Together, these developments make CKKS a framework for a much broader range of computations than its original approximate semantics suggest.

Large-Modulus Arithmetic. Extending these techniques to large integers raises a question of representation. Encoding an entire integer in a single real slot requires sufficient precision to distinguish adjacent values, with increasingly demanding requirements as the integer grows. Distributing the message across several smaller components reduces these precision requirements, but changes the cost of arithmetic, modular reduction, and ciphertext refreshing.

Several recent works explore this tradeoff. Building on the modular-reduction framework of Kim and Noh [KN25], Kim represents integers by smaller radix digits and uses functional bootstrapping to support integer operations [Kim25b]. Kim’s subsequent work improves multiplication latency for moduli of the form b^k with small b [Kim26]. Boneh and Kim instead use nested residue number systems to support arithmetic with large moduli [BK25]. Peikert *et al.* [PZZ26] introduce decomposed BFV (dBFV), which supports arbitrary integer moduli through digit decompositions across multiple BFV ciphertexts.

A complementary approach, which we follow here, chooses the plaintext modulus to exploit the underlying polynomial ring. It builds on the CLPX construction of Chen *et al.* [CLPX18], which replaces the integer plaintext modulus in BFV with $X - \beta$ to support arithmetic modulo $\beta^N + 1$ in a leveled setting, without ciphertext refreshing. The GBFV scheme of Geelen and Vercauteren [GV25] generalizes this construction to other polynomial plaintext moduli, enabling packed arithmetic modulo cyclotomic primes and efficient arithmetic bootstrapping. Kim subsequently showed how GBFV ciphertexts can be interpreted and bootstrapped using CKKS [Kim25a], establishing a direct connection between these constructions.

More recently, Gao and Zheng introduced GZ [GZ26], which supports arithmetic modulo powers of two with arithmetic bootstrapping requiring two CKKS bootstrapping calls per ciphertext, independently of the word size. Their construction uses a triangle encoding and linear transformations adapted to this representation. Its arithmetic-to-Boolean conversion requires a number of CKKS bootstrapping calls linear in the word size, but batching ciphertexts reduces this to an amortized constant cost per ciphertext. Concurrent work extends this line of research to large prime moduli [WMGZ26] and arbitrary plaintext moduli [LMWW26], and supports conversions to bit or digit representations with $\mathcal{O}(1)$ CKKS bootstrapping calls per ciphertext without amortization. These results demonstrate that large-modulus arithmetic can be supported efficiently within a CKKS-based framework. They also motivate studying how the choice of plaintext ring affects the transformations and refreshing procedures needed to sustain such arithmetic.

Our Contribution. We develop algorithms for exact arithmetic and refreshing in the polynomial lanes $(\mathbb{R}[X]/\langle X^n + 1 \rangle)^{N/n}$, natively supported by CKKS. We fix these ambient rings to simplify the changes of representation needed for refreshing. Following GBFV [GV25], we choose a polynomial plaintext modulus t and represent messages in $\mathbb{Z}[X]/\langle X^n + 1, t \rangle$ within each lane. The native ambient ring determines how we compute; the choice of t determines the finite ring in which the result is interpreted.

Our title, *Stay in Your Lane*, captures this design principle: the N/n polynomial lanes already present in CKKS provide a direct route to arithmetic over large finite rings. Since these lanes occur at intermediate stages of the conventional `CoeffToSlot` and `SlotToCoeff` transformations, we can reuse those stages for refreshing. “Leaving the lanes” to work in a different ambient ring can require additional transformations tailored to that ring. Staying within the existing lanes avoids this extra work and makes arithmetic and refreshing more efficient.

The choice of t , inherited from GBFV, retains some message-space flexibility within these fixed ambient rings. Useful choices depend on the quotient’s algebraic structure and the size of its polynomial representatives, so our approach is suited to applications that can choose from the supported message spaces.

For $t = X - \beta$, we obtain arithmetic modulo $\beta^n + 1$. Other binomial choices yield products of modular integer rings or extension fields such as $\mathbb{F}_{516}$, $\mathbb{F}_{174}$, and $\mathbb{F}_{3732}$, while non-binomial choices give specific 128-, 256-, 512-, and 1024-bit prime fields. A degree-32 lane can hold two $\mathbb{F}_{174}$ elements, giving 4096 elements per ciphertext when $N = 2^{16}$. Table 1 lists concrete choices and packing capacities, all supported by the same partial `CoeffToSlot` and `SlotToCoeff` framework.

We represent messages by bounded integer polynomials using GBFV’s flattening map [GV25]. Decoding rounds the recovered polynomial coefficients and reduces modulo t . We describe the resulting message spaces, analyze correctness and error growth of homomorphic arithmetic, and develop refreshing algorithms based on partial `CoeffToSlot` and `SlotToCoeff`. An implementation evaluates the cost of multiplication and refreshing and compares it with prior approaches. We are developing arithmetic-to-digit and digit-to-arithmetic conversion for a future version of this paper, as discussed in Section 9.

1.2 More Details on Our Method

Let $n \in \llbracket 2, N/2 \rrbracket$ be a power of two dividing the RLWE dimension N . Our ambient ring for one polynomial lane is $\mathcal{S}_n = \mathbb{R}[X]/\langle X^n + 1 \rangle$, with integer subring $\mathcal{R}_n = \mathbb{Z}[X]/\langle X^n + 1 \rangle$. We choose a

polynomial $t \in \mathbb{Z}[X]$ of degree less than n , coprime to $X^n + 1$ in $\mathbb{Q}[X]$ and use the finite message ring $\mathcal{A}_n = \mathcal{R}_n / \langle t \rangle$. Messages are represented by bounded integer polynomials in each lane, and N/n lanes are packed into one CKKS ciphertext. Polynomial arithmetic induces the desired arithmetic in $\mathcal{A}_n$; refreshing controls the size of the representatives and the approximation error.

From Quotients to Polynomial Representatives. We denote the quotient projection by $\text{Proj}_n : \mathcal{R}_n \rightarrow \mathcal{A}_n$. Because Proj_n preserves addition and multiplication, arithmetic on representatives gives the desired result modulo t . For example, if $t = X - \beta$ with an integer $\beta \geq 2$, evaluation at $X = \beta$ identifies $\mathcal{A}_n$ with $\mathbb{Z}_M = \mathbb{Z}/M\mathbb{Z}$ for $M = \beta^n + 1$. In this case, polynomial arithmetic performs modular integer arithmetic without carry propagation after each operation.

For encryption, we use GBFV’s *flattening operation* $P \mapsto t[P/t]_1$ [GV25] (also known as the hat encoder [CLPX18]) to choose a bounded representative. The coprimality assumption makes t invertible in $\mathcal{S}_n$, where division takes place, and $[\cdot]_1$ reduces each coefficient into $(-1/2, 1/2]$. The output differs from P by t times an integer polynomial, so it represents the same message, with coefficient magnitudes at most $\|t\|_1/2$, where $\|t\|_1$ is the sum of the absolute values of the coefficients of t in $\mathcal{R}_n$. For $t = X - \beta$, this gives the bound $(\beta + 1)/2$, which depends on β , not on M . More generally, this bound connects the choice of message ring to the size of its representatives and hence to error growth.

Packing and Homomorphic Arithmetic. The ring decomposition $\mathcal{S}_N \cong \mathcal{S}_n^{N/n}$ provides N/n independent polynomial lanes, each with ambient ring $\mathcal{S}_n$. It is implicit in the fast homomorphic Fourier transforms of Han *et al.* [HHC19]; these same polynomial rings also appear in NeuJeans [JPK⁺24] and PaCo [CS26]. Through this ring isomorphism, standard CKKS addition and multiplication perform polynomial arithmetic independently in each lane. A ciphertext thus has N/n lanes, each supporting arithmetic in $\mathcal{A}_n$. For $t = X - \beta$, each lane has a modulus of approximately $n \log_2 \beta$ bits. Decoding recovers the lane coefficients, rounds them to integers, and applies Proj_n . The rounding step recovers the exact coefficients when each error is less than $1/2$.

Refreshing. After removing the CKKS scale, a lane has the form $P + tI + e$, where P is the chosen bounded message representative, I is an integer *overflow polynomial*, and e is an approximation error. The term tI leaves the message modulo t unchanged, but growth of I amplifies error in subsequent multiplications. For sustained computation, refreshing must control the overflow I , restore ciphertext levels, and reduce the error e . We provide two refreshing algorithms. **LazyFlatten** (Algorithm 1) restores ciphertext levels and replaces I by a bounded overflow, without reducing the accumulated error. **Boot** (Algorithm 2) additionally reduces e .

Changes of Representation. These refreshing procedures require moving out of the polynomial lanes used for arithmetic: overflow control works in the degree- N coefficient representation, while error extraction requires access to individual coefficients in slots. When decomposed into their Fourier stages [HHC19], conventional **CoeffToSlot** and **SlotToCoeff** from CKKS natively pass through the (N/n) -lane representation that we use for our scheme. We can therefore start and stop at this intermediate representation, evaluating only the stages needed by each refreshing procedure. Figure 1 shows where our representation lies within these transformations.

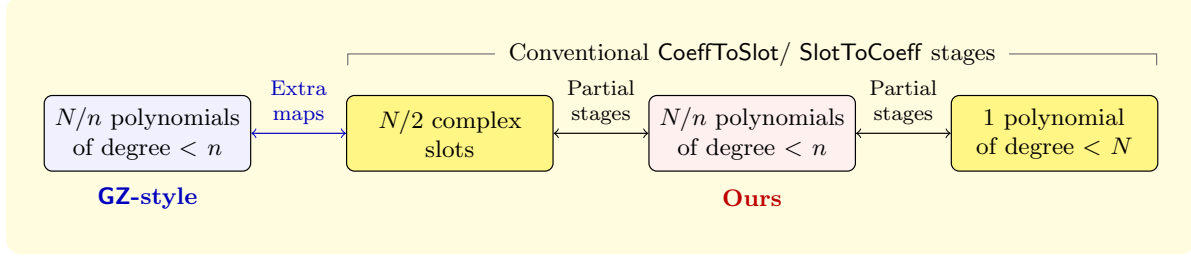


Fig. 1. Representation changes used in refreshing (schematic). Our arithmetic representation lies within the conventional CoeffToSlot/SlotToCoeff decomposition; GZ-style constructions require additional encoding-specific maps. Here, GZ-style refers to GZ [GZ26] and its arbitrary-modulus extensions [WMGZ26, LMWW26]. Only representation changes are shown.

Experimental Results. We compare with GZ because it also provides packed large-modulus arithmetic and arithmetic bootstrapping within CKKS, making it a natural baseline for evaluating the benefit of our choice of polynomial representation. Figure 2 compares arithmetic bootstrapping in GZ with our arithmetic bootstrapping (Algorithm 2) for $N = 2^{16}$ and $t = X - 2$, using N/n lanes in our construction and the same number of messages in GZ. GZ uses modulus 2^n , whereas we use $2^n + 1$. We implement both schemes in Lattigo [Lat24] and measure them on the same Apple M4 Pro using a single thread. Arithmetic bootstrapping is $2.2\times$ – $3.8\times$ faster for $n \in \llbracket 2^3, 2^6 \rrbracket$; our lazy flattening is $4.2\times$ – $16\times$ faster than GZ’s A2A-I for $n \in \llbracket 2^3, 2^7 \rrbracket$. These are the ranges in which the respective GZ procedures fit in the machine’s 24 GB of memory. Across $n \in \llbracket 2^3, 2^{14} \rrbracket$, our measured arithmetic bootstrapping time stays between 3.52 and 4.31 seconds, while lazy flattening falls from 0.89 to 0.07 seconds. The nearly constant arithmetic bootstrapping cost reflects a tradeoff: as the transformations between lanes and coefficients shorten, the transformation from slots back to lanes lengthens. For larger n , Figure 2 uses GZ’s published timings and a square-root extrapolation beyond $n = 2^8$. At $n = 2^{14}$, the extrapolation gives 97.2 seconds against our measured 3.69 seconds, an estimated $26\times$ speedup. This comparison uses different machines and extrapolates beyond GZ’s published measurements; Section 8 details its basis and limits.

The benefit extends to sustained arithmetic. Including lazy flattening and periodic arithmetic bootstrapping, the amortized cost computed from the measured operation times is 1.99 seconds per multiplication at $n = 2^3$ and 1.74 seconds at $n = 2^6$, compared with 6.62 and 15.0 seconds for our GZ port: speedups of $3.3\times$ and $8.6\times$, respectively. Arithmetic bootstrapping leaves a normalized error of about 2^{-35} , compared with 2^{-23} for the GZ port at its parameters. We also implement three extension-field message spaces; their refreshing times closely match those of modular-integer baselines $t = X - c$ with the same lane degree and constant c .

Comparison to Related Work. Relative to GZ and its extensions to broader classes of moduli [GZ26, WMGZ26, LMWW26], our choice of CKKS-native polynomial lanes avoids the additional encoding-specific maps needed to enter and leave the arithmetic representation. This simplifies refreshing at the cost of some flexibility in the message space. The distinction from CLPX [CLPX18] and GBFV [GV25] concerns where we take the plaintext quotient: CLPX and GBFV take a quotient of the integer RLWE ring $\mathcal{R}_N = \mathbb{Z}[X]/\langle X^N + 1 \rangle$, whereas we take quotients within small polynomial lanes $\mathcal{R}_n = \mathbb{Z}[X]/\langle X^n + 1 \rangle$ of degree n , chosen separately from the RLWE dimension N , ultimately leading to more flexibility in the choice of message ring. This distinction also applies to Kim’s

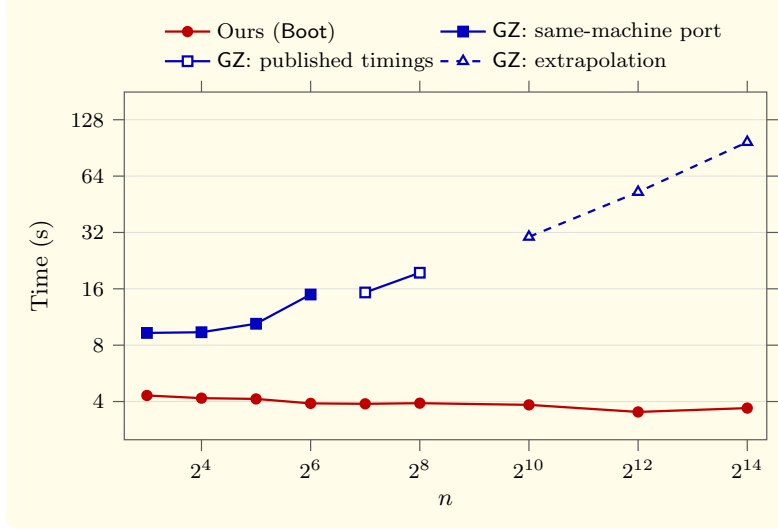


Fig. 2. Arithmetic bootstrapping time per ciphertext as a function of n , at $N = 2^{16}$: our **Boot** versus GZ’s A2A. Our timings (circles) are measured throughout. For GZ, filled squares at $n \leq 2^6$ are measurements of our port in the same library, **Lattigo**, on the same Apple M4 Pro with one thread (medians of ten runs). Open squares at $n = 2^7$ and $n = 2^8$ are published x86/HEXL timings from [GZ26]; triangles on the dashed line at $n > 2^8$ are estimates from a square-root fit to their published timings for $n \in [2^3, 2^8]$ (Section 8). Thus the wider comparison combines different machines and extrapolated data. We use $t = X - 2$. Our ciphertexts have N/n lanes, each carrying one message modulo $2^n + 1$; GZ packs the same number of messages modulo 2^n . Both axes are logarithmic. Memory limits restrict our measurement of their arithmetic bootstrapping to $n \leq 2^6$.

CKKS-based arithmetic bootstrapping of CLPX and GBFV [Kim25a], which refreshes ciphertexts while retaining their respective plaintext spaces.

1.3 Paper Organization

Section 2 introduces our notation, and Section 3 reviews the CKKS operations used in our construction. Section 4 defines lifting and projection between the message ring $\mathcal{A}_n$ and integer polynomials in $\mathcal{R}_n$, and establishes the bounds needed for homomorphic arithmetic. Section 5 gives concrete choices of message space, including modular integer rings, finite fields, and their products; Table 1 summarizes selected parameters. Section 6 defines encryption and homomorphic operations and bounds their overflow and error growth. Section 7 presents the refreshing algorithms **LazyFlatten** and **Boot** and states their correctness guarantees. Section 8 evaluates running time and precision and compares the cost of refreshing with GZ [GZ26]. Finally, Section 9 outlines our ongoing work on conversions between arithmetic and digit representations and broader comparisons with recent and concurrent schemes.

2 Notation

Sets. We write $\mathbb{Z}$, $\mathbb{Q}$, $\mathbb{R}$ and $\mathbb{C}$ for the sets of integers, rational numbers, real numbers and complex numbers, respectively. Given an integer $q \geq 1$, we denote by $\mathbb{Z}_q = \mathbb{Z}/q\mathbb{Z}$ the ring of integers modulo q . For two integers $p \leq q$, we let $\llbracket p, q \rrbracket = \{p, p+1, \dots, q\}$, and for an integer $n \geq 1$, we write $\llbracket n \rrbracket = \llbracket 0, n-1 \rrbracket$.

Modular Reduction and Rounding. For a real number $x \in \mathbb{R}$ and an integer $q \geq 1$, we write $[x]_q$ for the unique real number $y \in (-q/2, q/2]$ such that $x - y$ is an integer multiple of q . We write $\lfloor x \rfloor = x - [x]_1$ for rounding to the nearest integer, with ties rounded downwards. Both notations extend entrywise to tuples and coefficientwise to polynomials. Given a ring R and two elements $a, b \in R$, we write $a \bmod b$ for the projection of a onto the quotient $R/\langle b \rangle$. For $a, a' \in R$, we write $a \equiv_b a'$ if $a \bmod b = a' \bmod b$.

Norms. We denote the 1-norm, 2-norm, and infinity norm of a complex vector $\mathbf{z} \in \mathbb{C}^n$ as follows:

$$\|\mathbf{z}\|_1 = \sum_{i=0}^{n-1} |z_i|, \quad \|\mathbf{z}\|_2 = \sqrt{\sum_{i=0}^{n-1} |z_i|^2}, \quad \|\mathbf{z}\|_\infty = \max_{i \in [n]} |z_i|,$$

The same norms apply to complex polynomials by considering their coefficient vector; for instance, if $P = \sum_{i=0}^{n-1} p_i X^i$, then $\|P\|_1 = \sum_{i=0}^{n-1} |p_i|$. For a tuple of vectors or polynomials, these norms are taken on the concatenation of the vectors or coefficient vectors, respectively. For instance, $\|(2 + 5X, 4 - 2iX^2)\|_2 = \|(2, 5, 4, -2i)\|_2 = 7$.

Matrices. The identity matrix of size $m \times m$ is denoted by $\mathbf{I}_m$. For a vector $\mathbf{z} \in \mathbb{C}^n$, we write $\text{diag}(\mathbf{z}) \in \mathbb{C}^{n \times n}$ for the diagonal matrix with $\mathbf{z}$ on the diagonal. Given an $n \times n$ matrix $\mathbf{M}$, we denote by $\mathbf{I}_m \otimes \mathbf{M}$ the block-diagonal matrix of size $(mn) \times (mn)$ with $\mathbf{M}$ repeated m times on the diagonal. For a complex matrix $\mathbf{M} \in \mathbb{C}^{m \times n}$, we denote its conjugate transpose by $\mathbf{M}^* \in \mathbb{C}^{n \times m}$.

Permutations. For a power of two $n = 2^k$, we denote by $\text{br}_n : [n] \rightarrow [n]$ the *bit reversal permutation*, which reverses the k bits of an index. For $x \in [n]$ with binary expansion $x = \sum_{i=0}^{k-1} x_i 2^i$, we have $\text{br}_n(x) = \sum_{i=0}^{k-1} x_i 2^{k-1-i}$.

Miscellaneous. For a finite nonempty set A , we write $a \leftarrow A$ to denote sampling a uniformly at random from A . We denote the imaginary unit by $i \in \mathbb{C}$. For an integer $n \geq 1$, we write $\zeta_n = \exp(2\pi i/n) \in \mathbb{C}$ for the n -th primitive root of unity with argument $2\pi/n$. Given a map $f : A \rightarrow B$, we abuse notation and also write $f : A^n \rightarrow B^n$ for the componentwise map $(a_i)_{i \in [n]} \mapsto (f(a_i))_{i \in [n]}$. For vectors $\mathbf{z}, \mathbf{w} \in \mathbb{C}^n$, we write $\mathbf{z} \odot \mathbf{w} = (z_i w_i)_{i \in [n]}$ for their componentwise (Hadamard) product. For $\mathbf{z} \in \mathbb{C}^n$ and $j \in \mathbb{Z}$, we write $\text{Rot}_j(\mathbf{z}) = (z_{i+j})_{i \in [n]}$ for the cyclic rotation of $\mathbf{z}$ by j positions, with indices interpreted cyclically with period n .

3 Background on CKKS

We recall the key elements of the CKKS scheme [CKKS17], which is based on RLWE encryption [LPR10, SSTX09]. We do not recall security notions here; we refer to [CKKS17, LM21, LM-SWS22] for definitions and security analyses of approximate homomorphic encryption. For ease of explanation, we use a simplified, non-RNS model; the RNS variant is described in [CHK⁺19].

3.1 Encoding and Decoding

We first introduce the following *evaluation map*:

$$\text{eval}_n : \mathcal{S}_n \rightarrow \mathbb{C}^{n/2}, \quad \text{eval}_n(P) = P\left(\left(\zeta_{2n}^{5^i}\right)_{i \in \llbracket n/2 \rrbracket}\right).$$

For a fixed scale $\Delta \gg 0$, the encoding and decoding maps are given by:

$$\begin{aligned} \text{Ecd}_{\mathbb{C}} : \mathbb{C}^{N/2} &\rightarrow \mathcal{R}_N, & \text{Ecd}_{\mathbb{C}}(\mathbf{z}) &= \lfloor \Delta \text{eval}_N^{-1}(\mathbf{z}) \rfloor, \\ \text{Dcd}_{\mathbb{C}} : \mathcal{R}_N &\rightarrow \mathbb{C}^{N/2}, & \text{Dcd}_{\mathbb{C}}(m) &= \text{eval}_N\left(\frac{m}{\Delta}\right). \end{aligned}$$

These two maps allow us to view an element of $\mathcal{R}_N$ as a vector of $N/2$ complex numbers. They are approximate inverses of each other.

3.2 Encryption and Decryption

For an integer $q \geq 1$, let $\mathcal{R}_{N,q} \subseteq \mathcal{R}_N$ denote the set of polynomials in $\mathcal{R}_N$ with coefficients in $(-q/2, q/2]$. We fix a base modulus q_0 and a maximal level $L \geq 1$, using the scale Δ defined above. The *CKKS modulus chain* is the sequence $q_0 < \dots < q_L$, where $q_\ell = q_0 \Delta^\ell$ for $\ell \in \llbracket 0, L \rrbracket$. A CKKS ciphertext at level $\ell \in \llbracket 0, L \rrbracket$ is a pair $\mathbf{ct} = (\text{ct}_0, \text{ct}_1) \in \mathcal{R}_{N,q_\ell}^2$. For a secret key $s \in \mathcal{R}_N$ and an input polynomial $m \in \mathcal{R}_N$, fresh secret-key encryption $\mathbf{ct} \leftarrow \text{Enc}_s(m)$ is performed as follows: we sample $e_{\text{enc}} \in \mathcal{R}_N$ from the chosen RLWE error distribution [LPR10] and let $\text{Enc}_s(m) = (\text{ct}_0, \text{ct}_1) \in \mathcal{R}_{N,q_L}^2$, where $\text{ct}_0 \leftarrow \mathcal{R}_{N,q_L}$ and $\text{ct}_1 \leftarrow [m - \text{ct}_0 s + e_{\text{enc}}]_{q_L}$. Decryption is defined by:

$$\text{Dec}_{s,\ell} : \mathcal{R}_{N,q_\ell}^2 \rightarrow \mathcal{R}_{N,q_\ell}, \quad \text{Dec}_{s,\ell}(\mathbf{ct}) = [\text{ct}_0 s + \text{ct}_1]_{q_\ell} \in \mathcal{R}_{N,q_\ell}.$$

For a general ciphertext, we interpret decryption as $\text{Dec}_{s,\ell}(\mathbf{ct}) = \Delta P + e$, where $P \in \mathcal{S}_N$ is the message polynomial and $e \in \mathcal{S}_N$ is the accumulated error term. We call $\mathbf{z} = \text{eval}_N(P) \in \mathbb{C}^{N/2}$ the *slot vector* of $\mathbf{ct}$, and its entries the *slot values*. These are the ideal values; decoding returns $\mathbf{z} + \text{eval}_N(e)/\Delta$. Throughout, an error bound refers to the additive error in the decryption relation, before division by the ciphertext scale. Thus, an error e at scale Δ contributes e/Δ to the decoded polynomial, or $\text{eval}_N(e)/\Delta$ to the decoded slots. Encryption acts on the rounded encoding $\text{Ecd}_{\mathbb{C}}(\mathbf{z}) = \lfloor \Delta P \rfloor$. Thus, e accounts for the rounding discrepancy $\text{Ecd}_{\mathbb{C}}(\mathbf{z}) - \Delta P = -[\Delta P]_1$, the noise introduced by RLWE encryption, and the propagation of these errors and introduction of new errors during homomorphic evaluation.

3.3 Basic Homomorphic Operations

Consider ciphertexts $\mathbf{ct}, \mathbf{ct}' \in \mathcal{R}_{N,q_\ell}^2$ at level $\ell \in \llbracket 0, L \rrbracket$ under the same secret key $s \in \mathcal{R}_N$, satisfying $\text{Dec}_{s,\ell}(\mathbf{ct}) = \Delta P + e$ and $\text{Dec}_{s,\ell}(\mathbf{ct}') = \Delta P' + e'$ with message polynomials $P, P' \in \mathcal{S}_N$ (for instance, $P = \text{eval}_N^{-1}(\mathbf{z})$ and $P' = \text{eval}_N^{-1}(\mathbf{z}')$ for some $\mathbf{z}, \mathbf{z}' \in \mathbb{C}^{N/2}$) and error terms $e, e' \in \mathcal{S}_N$. We further fix a plaintext polynomial $m \in \mathcal{R}_N$ (for instance, $m = \text{Ecd}_{\mathbb{C}}(\mathbf{w})$ for some $\mathbf{w} \in \mathbb{C}^{N/2}$). The unbracketed decryption equalities below assume that all coefficients on the right-hand side lie in the centered interval for the output modulus. More details are provided in [CKKS17, CHK⁺18].

- *Key switching.* We can homomorphically change the secret key from s to another secret key $s' \in \mathcal{R}_N$ by computing $\mathbf{ct}_{\text{ks}} \leftarrow \text{KeySwitch}_{s' \leftarrow s}(\mathbf{ct})$. The output satisfies $\text{Dec}_{s',\ell}(\mathbf{ct}_{\text{ks}}) = \Delta P + e + e_{\text{ks}}$, where $e_{\text{ks}} \in \mathcal{S}_N$ is a key-switching error. This procedure requires a key-switching key $\text{swk}_{s' \leftarrow s}$.

- *Plaintext-ciphertext addition.* We compute $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}(m, \mathbf{ct}) = [(0, m) + \mathbf{ct}]_{q_\ell}$, which satisfies $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{add}}) = m + \Delta P + e$. Subtraction works similarly.
- *Ciphertext-ciphertext addition.* We compute $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}(\mathbf{ct}, \mathbf{ct}') = [\mathbf{ct} + \mathbf{ct}']_{q_\ell}$, which satisfies $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{add}}) = \Delta(P + P') + e + e'$. Subtraction works similarly.
- *Rescaling.* If $\ell \geq 1$, we compute $\mathbf{ct}_{\text{rs}} \leftarrow \text{Rescale}(\mathbf{ct}) = \lfloor \mathbf{ct}/\Delta \rfloor_{q_{\ell-1}}$. The output satisfies $\text{Dec}_{s,\ell-1}(\mathbf{ct}_{\text{rs}}) = P + e/\Delta + e_{\text{rs}}$, where $e_{\text{rs}} \in \mathcal{S}_N$ is a rescaling error.
- *Modulus raising.* For $\ell, \ell' \in \llbracket 0, L \rrbracket$ with $\ell \leq \ell'$, we compute $\mathbf{ct}_{\text{mr}} \leftarrow \text{ModRaise}_{\ell' \leftarrow \ell}(\mathbf{ct})$ by reinterpreting the centered coefficients of both ciphertext components modulo $q_{\ell'}$. The output satisfies $\text{Dec}_{s,\ell'}(\mathbf{ct}_{\text{mr}}) = \Delta P + e + q_\ell I$ for some $I \in \mathcal{R}_N$.
- *Plaintext-ciphertext multiplication.* We compute $\mathbf{ct}_{\text{mul}} \leftarrow \text{Mul}(m, \mathbf{ct}) = [m \cdot \mathbf{ct}]_{q_\ell}$, which satisfies $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{mul}}) = \Delta m P + me$. This operation requires no key-switching key and consumes no level.
- *Ciphertext-ciphertext multiplication.* We compute $\mathbf{ct}_{\text{mul}} \leftarrow \text{Mul}(\mathbf{ct}, \mathbf{ct}')$ by tensoring the ciphertexts and relinearizing with $\text{swk}_{s \leftarrow s^2}$. The output satisfies $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{mul}}) = \Delta^2 P P' + \Delta(Pe' + P'e) + ee' + e_{\text{ks}}$. This operation is typically followed by rescaling to reduce the scale back to Δ .
- *Conjugation.* Let τ be the automorphism $X \mapsto X^{-1} = -X^{N-1}$. Applying τ to both ciphertext components and key-switching from $\tau(s)$ back to s gives $\mathbf{ct}_{\text{conj}} \leftarrow \text{Conj}(\mathbf{ct})$, with $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{conj}}) = \Delta \tau(P) + \tau(e) + e_{\text{ks}}$, where $\text{eval}_N(\tau(P)) = \bar{\mathbf{z}}$ if $\text{eval}_N(P) = \mathbf{z}$. This procedure requires $\text{swk}_{s \leftarrow \tau(s)}$.
- *Rotation.* For $j \in \llbracket N/2 \rrbracket$, let σ_j be the automorphism $X \mapsto X^{5^j}$. Applying σ_j to both ciphertext components and key-switching from $\sigma_j(s)$ to s gives $\mathbf{ct}_{\text{rot}} \leftarrow \text{Rot}_j(\mathbf{ct})$, with $\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{rot}}) = \Delta \sigma_j(P) + \sigma_j(e) + e_{\text{ks}}$, where $\text{eval}_N(\sigma_j(P)) = \text{Rot}_j(\mathbf{z})$ if $\text{eval}_N(P) = \mathbf{z}$. This procedure requires $\text{swk}_{s \leftarrow \sigma_j(s)}$. For $j = 0$, we can simply return $\mathbf{ct}$, with no key switching or added error.

3.4 Homomorphic Linear Transformations

For a matrix $\mathbf{M} \in \mathbb{C}^{(N/2) \times (N/2)}$, define its j -th diagonal as $\text{Diag}_j(\mathbf{M}) = (M_{i,i+j})_{i \in \llbracket N/2 \rrbracket}$, with column indices taken modulo $N/2$, and its diagonal support as $\text{DiagSupp}(\mathbf{M}) = \{j \in \llbracket N/2 \rrbracket : \text{Diag}_j(\mathbf{M}) \neq \mathbf{0}\}$. A matrix is *diagonally sparse* if this support is small. The *diagonal method* [HS14] gives $\mathbf{Mz} = \sum_j \text{Diag}_j(\mathbf{M}) \odot \text{Rot}_j(\mathbf{z})$, with the sum taken over $\text{DiagSupp}(\mathbf{M})$. Let $\text{Ecd}_{\mathbb{C}}(\mathbf{M})$ denote the encoded nonzero diagonals. A direct homomorphic implementation is:

$$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{M}), \mathbf{ct}) = \text{Rescale} \left(\left[\sum_j \text{Mul}(\text{Ecd}_{\mathbb{C}}(\text{Diag}_j(\mathbf{M})), \text{Rot}_j(\mathbf{ct})) \right]_{q_\ell} \right);$$

the matrix $\mathbf{M}$ is then homomorphically applied to the slots of $\mathbf{ct}$. The direct method uses $|\text{DiagSupp}(\mathbf{M})|$ plaintext-ciphertext multiplications, the same number of rotations, and one rescaling. If the diagonal support lies in an arithmetic progression modulo $N/2$ of length D , then the *baby-step giant-step evaluation* [HS14] uses $\mathcal{O}(D)$ plaintext-ciphertext multiplications and $\mathcal{O}(\sqrt{D})$ rotations. Further, some matrices $\mathbf{M}$ can be factored into a product of diagonally sparse matrices, as is for instance the case for discrete Fourier transforms [HHC19], and in such cases, one can further reduce operation counts at the cost of additional levels. The notation LinTrans leaves the choice of implementation implicit; for a factored implementation, $\text{Ecd}_{\mathbb{C}}(\mathbf{M})$ denotes the encoded nonzero diagonals of the factors.

For a ciphertext $\mathbf{ct}$ at level ℓ with $\text{Dec}_{s,\ell}(\mathbf{ct}) = \Delta \text{eval}_N^{-1}(\mathbf{z}) + e$, let $\mathbf{ct}_{\text{lin}} \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{M}), \mathbf{ct})$ and let $\tilde{\ell}$ be its output level (with $\tilde{\ell} = \ell - 1$ for the direct method). Provided sufficient levels are available, each implementation yields:

$$\text{Dec}_{s,\tilde{\ell}}(\mathbf{ct}_{\text{lin}}) = \Delta \text{eval}_N^{-1}(\mathbf{Mz}) + \text{eval}_N^{-1}(\mathbf{M} \text{eval}_N(e)) + e_{\text{lt}},$$

provided the right-hand side has coefficients in $(-q_{\tilde{\ell}}/2, q_{\tilde{\ell}}/2]$. The second term propagates the input error through the same linear map. The additional error e_{lt} depends on the chosen implementation and collects diagonal-encoding, key-switching, and rescaling errors, including their propagation through subsequent factors. More details are for instance provided in [KPK⁺22].

3.5 Partial CoeffToSlot and SlotToCoeff

The complex slots of a CKKS ciphertext can be grouped into independent polynomial lanes. For a power of two $d \in \llbracket 2, N \rrbracket$, applying eval_d^{-1} to each block of $d/2$ slots gives N/d polynomials in $\mathcal{S}_d$, with addition and multiplication acting independently in each lane. For powers of two $d, d' \in \llbracket 2, N \rrbracket$, the corresponding change of interpretation is given by the *packing isomorphism*:

$$\Phi_{d' \leftarrow d} = \text{eval}_{d'}^{-1} \circ \text{eval}_d : \mathcal{S}_d^{N/d} \rightarrow \mathcal{S}_{d'}^{N/d'}.$$

Here, we evaluate each input polynomial, regroup the resulting slots into blocks of $d'/2$, and apply $\text{eval}_{d'}^{-1}$ to each block. The maps $\Phi_{d' \leftarrow d}$ and $\Phi_{d \leftarrow d'}$ are mutual inverses and preserve addition and multiplication. In particular, $\Phi_{N \leftarrow d}$ packs the polynomial lanes into the native CKKS plaintext ring $\mathcal{S}_N$, preserving the underlying slot vector.

Refreshing requires the lane coefficients in other representations: collected in a single polynomial for overflow control, or exposed as real and imaginary slot components for error extraction. This motivates a second family of maps that rearrange the coefficients.

For powers of two $d, d' \in \llbracket 2, N \rrbracket$ with $d \leq d'$, the following map packs each group of d'/d polynomials in $\mathcal{S}_d$ into a single polynomial in $\mathcal{S}_{d'}$, preserving their coefficient values:

$$\kappa_{d' \leftarrow d} : \mathcal{S}_d^{N/d} \rightarrow \mathcal{S}_{d'}^{N/d'}, \quad \kappa_{d' \leftarrow d} \left((P_i)_{i \in \llbracket N/d \rrbracket} \right) = \left(\sum_{j=0}^{d'/d-1} X^{\text{br}_{d'/d}(j)} P_{id'/d+j} (X^{d'/d}) \right)_{i \in \llbracket N/d' \rrbracket}.$$

We also set $\kappa_{d \leftarrow d'} = \kappa_{d' \leftarrow d}^{-1} : \mathcal{S}_{d'}^{N/d'} \rightarrow \mathcal{S}_d^{N/d}$. These maps only permute coefficients, so they are linear isomorphisms that preserve coefficient norms. The bit reversal makes these maps decompose into pairwise packing steps, as in Theorem 1 below. At the endpoints, $\kappa_{2 \leftarrow N}$ and $\kappa_{N \leftarrow 2}$ recover the conventional CoeffToSlot and SlotToCoeff operations. The intermediate maps $\kappa_{d' \leftarrow d}$ and $\kappa_{d \leftarrow d'}$ are sometimes called *partial SlotToCoeff* and *partial CoeffToSlot*, respectively (e.g., [CS26]).

On the slot vector, each pairwise packing step becomes a sparse *butterfly* transformation. For powers of two $d, d' \in \llbracket 2, N \rrbracket$ with $d < d'$, we define $\mathbf{U}_{d' \leftarrow d} = \mathbf{U}_{d'} \cdot \mathbf{U}_{d'/2} \cdots \mathbf{U}_{2d}$ and $\mathbf{U}_{d \leftarrow d'} = (d/d') \cdot \mathbf{U}_{d' \leftarrow d}^*$. For $d = d'$, we set $\mathbf{U}_{d \leftarrow d} = \mathbf{I}_{N/2}$; this identity transformation requires no homomorphic operations and consumes zero levels. Here, for every power of two $k \in \llbracket 4, N \rrbracket$, the matrix $\mathbf{U}_k$ is defined by:

$$\mathbf{U}_k = \mathbf{I}_{N/k} \otimes \begin{pmatrix} \mathbf{I}_{k/4} & \mathbf{W}_{k/4} \\ \mathbf{I}_{k/4} & -\mathbf{W}_{k/4} \end{pmatrix} \in \mathbb{C}^{(N/2) \times (N/2)}, \quad \mathbf{W}_{k/4} = \text{diag} \left((\zeta_{2k}^{5i})_{i \in \llbracket k/4 \rrbracket} \right) \in \mathbb{C}^{(k/4) \times (k/4)}.$$

Theorem 1 (Partial CoeffToSlot and SlotToCoeff). *For all powers of two $d, d' \in \llbracket 2, N \rrbracket$, the following identity holds as maps $\mathbb{C}^{N/2} \rightarrow \mathbb{C}^{N/2}$:*

$$\text{eval}_{d'} \circ \kappa_{d' \leftarrow d} \circ \text{eval}_d^{-1} = \mathbf{U}_{d' \leftarrow d}.$$

Proof. This follows by composing the butterfly factorizations in [HHC19, Section IV-B, Lemma 5]. $\square$

By the theorem, if a ciphertext represents $\mathbf{P} \in \mathcal{S}_d^{N/d}$ via $\Phi_{N \leftarrow d}$, applying $\mathbf{U}_{d' \leftarrow d}$ to its slots yields a ciphertext representing $\kappa_{d' \leftarrow d}(\mathbf{P})$ via $\Phi_{N \leftarrow d'}$. Taking $d' = N$ collects the lane coefficients into a single polynomial; taking $d' = 2$ exposes them in the real and imaginary slot components. The reverse transformations restore the lanes, using only the Fourier stages between the source and target degrees.

For $d < d'$, the $\log_2(d'/d)$ butterfly stages can be grouped into fewer, denser transformations and evaluated using the *diagonal method* with *baby-step giant-step* [HS14, HHC19]. This gives a tradeoff between multiplicative depth and operation count: for any integer $\rho \in \llbracket 1, \log_2(d'/d) \rrbracket$, both $\mathbf{U}_{d' \leftarrow d}$ and $\mathbf{U}_{d \leftarrow d'}$ can be evaluated in ρ multiplicative levels using $\mathcal{O}(\rho \cdot (d'/d)^{1/\rho})$ plaintext–ciphertext multiplications and $\mathcal{O}(\rho \cdot (d'/d)^{1/(2\rho)})$ rotations [HS14, HHC19].

4 Lifting and Projection

We now describe how to represent messages by bounded integer polynomials and recover them by projection. For completeness, we provide in Appendix A the proofs of the results in this section. Fix a power of two $n \in \llbracket 2, N/2 \rrbracket$. We exclude $n = N$, for which the message space is a quotient of the full integer RLWE ring $\mathcal{R}_N$, as treated directly by GBFV [GV25] and Kim’s CKKS-based arithmetic bootstrapping [Kim25a]. Let $t \in \mathbb{Z}[X]$ have degree less than n and be coprime to $X^n + 1$ in $\mathbb{Q}[X]$, and define the resulting quotient $\mathcal{A}_n = \mathcal{R}_n / \langle t \rangle$. We also write t for its image in $\mathcal{R}_n$ and $\mathcal{S}_n$. The degree restriction entails no loss of generality: replacing t by its remainder modulo $X^n + 1$ preserves these images and the quotient $\mathcal{A}_n$. The coprimality assumption ensures that t is invertible in $\mathcal{S}_n$, where all divisions by t below take place. The full message space is $\mathcal{A}_n^{N/n}$, with one element in each of the N/n polynomial lanes. It therefore suffices to define lifting and projection for one lane. The quotient projection $\text{Proj}_n : \mathcal{R}_n \rightarrow \mathcal{A}_n$ recovers the message represented by an integer polynomial; its kernel is $\langle t \rangle$. For the reverse direction, we choose a bounded representative using GBFV’s *flattening map* [GV25] (also known as the *hat encoder* in [CLPX18]):

$$\text{Flatten}_n : \mathcal{R}_n \rightarrow \mathcal{R}_n, \quad \text{Flatten}_n(P) = t \cdot \left[\frac{P}{t} \right]_1.$$

Although flattening involves division in $\mathcal{S}_n$, its output is again an integer polynomial.

Lemma 1 (Well-definedness of the flattening map). *The map $\text{Flatten}_n : \mathcal{R}_n \rightarrow \mathcal{R}_n$ is well-defined.*

To use this map for lifting, we need it to preserve the message and give the same output for every representative of that message. The following properties establish both facts, as well as the stability of the output under repeated flattening.

Proposition 1 (Basic properties of the flattening map). *The map $\text{Flatten}_n : \mathcal{R}_n \rightarrow \mathcal{R}_n$ satisfies the following properties:*

- (a) *For every $P \in \mathcal{R}_n$, we have $(\text{Proj}_n \circ \text{Flatten}_n)(P) = \text{Proj}_n(P)$.*
- (b) *If $P, Q \in \mathcal{R}_n$ are such that $\text{Proj}_n(P) = \text{Proj}_n(Q)$, then $\text{Flatten}_n(P) = \text{Flatten}_n(Q)$.*
- (c) *$\text{Flatten}_n \circ \text{Flatten}_n = \text{Flatten}_n$.*

Since flattening is constant on each residue class, it defines a canonical representative for every message. We use it to define the lifting map:

$$\text{Lift}_n : \mathcal{A}_n \rightarrow \mathcal{R}_n, \quad \text{Lift}_n(a) = \text{Flatten}_n(P),$$

where $P \in \mathcal{R}_n$ is any polynomial such that $\text{Proj}_n(P) = a$. Projection recovers the original message from this representative, giving the basic correctness identity for encoding and decoding.

Proposition 2 (Projecting after lifting). *The map $\text{Proj}_n : \mathcal{R}_n \rightarrow \mathcal{A}_n$ is a left inverse of the encoding map $\text{Lift}_n : \mathcal{A}_n \rightarrow \mathcal{R}_n$:*

$$\text{Proj}_n \circ \text{Lift}_n = \text{id}_{\mathcal{A}_n}.$$

For homomorphic computation, we also need control over the size of the chosen representatives. The next bounds hold uniformly over all messages and relate their coefficient magnitudes to the choice of t .

Proposition 3 (Lifting bounds). *For every $a \in \mathcal{A}_n$, it holds that:*

$$\left\| \frac{\text{Lift}_n(a)}{t} \right\|_\infty \leq \frac{1}{2}, \quad \|\text{Lift}_n(a)\|_\infty \leq \frac{\|t\|_1}{2}.$$

Arithmetic need not preserve this canonical form. Given $a \in \mathcal{A}_n$, any polynomial $P \in \mathcal{R}_n$ such that $\text{Proj}_n(P) = a$ can be written as $P = \text{Lift}_n(a) + tI$ for some $I \in \mathcal{R}_n$; we call I the *overflow polynomial* [LMWW26]. The term tI records the departure from the canonical representative without changing the message. The final result shows that addition and multiplication preserve this description: each produces a representative of the desired result together with a new overflow polynomial.

Proposition 4 (Lifting respects arithmetic operations). *Let $a, a' \in \mathcal{A}_n$ and let $P = \text{Lift}_n(a) + tI$ and $P' = \text{Lift}_n(a') + tI'$ for some $I, I' \in \mathcal{R}_n$. Then $P + P' = \text{Lift}_n(a + a') + tI_{\text{add}}$ and $PP' = \text{Lift}_n(aa') + tI_{\text{mul}}$ for some $I_{\text{add}}, I_{\text{mul}} \in \mathcal{R}_n$.*

We can therefore evaluate arithmetic on integer polynomial representatives while tracking the resulting overflow. Under CKKS, an additional approximation error is present; the refreshing procedures will control both contributions.

5 Concrete Candidates for Message Space

The choice of t determines the quotient $\mathcal{A}_n = \mathcal{R}_n / \langle t \rangle$ within each native polynomial lane. We now give concrete choices yielding modular integer rings, finite fields, and products of such rings. Each result specifies its own assumptions on t . The coefficient bound $\|\text{Lift}_n(a)\|_\infty \leq \|t\|_1/2$ from Proposition 3 relates these choices to the size of the representatives.

In the linear case of the next result, each native polynomial lane uses the quotient construction of CLPX [CLPX18]. Each ciphertext has N/n lanes, with one modular integer per lane in the linear case. Larger values of r give r modular integers per lane, for a total of rN/n integers across the same N/n lanes.

Proposition 5 (Residue-ring decomposition). *Let $t = X^r - \beta^r$, where $r \in \llbracket 1, n/2 \rrbracket$ is a power of two and $\beta \geq 2$ is an integer. If $r = 1$ or if β is even, we have an isomorphism of rings $\mathcal{A}_n \cong \mathbb{Z}_M^r$ for $M = \beta^n + 1$.*

When M is prime, this decomposition is also a special case of [GV25, Lemma 9 and Equation (4)]. The statement above allows composite M ; we give a proof covering this case in Appendix B. For prime quotients, GBFV also identifies when the factors are nontrivial extension fields. The following proposition specializes that decomposition to our native polynomial rings.

The next proposition drops the assumption that the constant term is a perfect r -th power, writing it c ; for $c = \beta^r$ it is Proposition 5 again, since $c^{n/r} = \beta^n$.

Proposition 6 (Finite-field decomposition). *Let $t = X^r - c$, where $r \in \llbracket 1, n/2 \rrbracket$ is a power of two and $c \geq 2$ is an integer. If $M = c^{n/r} + 1$ is prime, then $\mathcal{A}_n \cong \mathbb{F}_{M^u}^{r/u}$, where $u = \text{ord}_{2n}(M)$.*

Proof. Apply [GV25, Lemma 9 and Equation (4)] with $m = 2n$, $k = r$, and $b = c$. $\square$

For non-binomial choices of t , we use the following prime-field criterion.

Proposition 7 (Prime-field quotients). *Let $t \in \mathbb{Z}[X]$ have degree less than n , and define $p = \left| \prod_{j=0}^{n-1} t(\zeta_{2n}^{2j+1}) \right| \in \mathbb{Z}$. If p is prime, then $\mathcal{A}_n \cong \mathbb{F}_p$.*

Proof. This follows from [GV25, Lemmas 2 and 3]. $\square$

Table 1 gives twelve concrete choices at RLWE dimension $N = 2^{16}$, together with the total number of independent elements carried by the N/n lanes of one ciphertext. They cover two composite integer moduli with different packing densities, four extension fields, and four large prime fields. These are algebraic examples of our framework; the timing experiments in Section 8 cover only selected binomial cases. We also report $\|t\|_1$ and $\|1/t\|_1$, which enter the bounds on lifted representatives and on error propagation in lazy flattening.

6 Our Framework within CKKS

Throughout this section, $t \in \mathbb{Z}[X]$ has degree less than n and is coprime to $X^n + 1$ in $\mathbb{Q}[X]$, as in Section 4. Error growth in the refreshing algorithms below depends on both $\|t\|_1$ and $\|1/t\|_1$, and it is thus beneficial to choose t such that both quantities remain small. We now explain how we encrypt messages in N/n lanes, each with message ring $\mathcal{A}_n$, using the CKKS scheme. The idea is that CKKS natively supports approximate arithmetic in $\mathcal{S}_N$, and we use this to perform approximate arithmetic in $\mathcal{S}_n^{N/n}$, which in turn contains the discrete subring $\mathcal{R}_n^{N/n}$, the target ring of the map Lift_n . We first describe the ciphertext representation and conditions for correct recovery, then show how fresh encryption and homomorphic operations fit this framework. We provide proofs for all statements in this section in Appendix C.

6.1 Ciphertexts

We use the packing isomorphisms $\Phi_{N \leftarrow n} : \mathcal{S}_n^{N/n} \rightarrow \mathcal{S}_N$ and $\Phi_{n \leftarrow N} : \mathcal{S}_N \rightarrow \mathcal{S}_n^{N/n}$ introduced in Section 3.5. In our proposed scheme, a CKKS ciphertext encrypting $\mathbf{a} \in \mathcal{A}_n^{N/n}$ at level $\ell \in \llbracket 0, L \rrbracket$ with respect to the secret key $s \in \mathcal{R}_N$ is a tuple $\mathbf{ct} \in \mathcal{R}_{N, q\ell}^2$ such that:

$$\text{Dec}_{s, \ell}(\mathbf{ct}) = \Phi_{N \leftarrow n}(\Delta \text{Lift}_n(\mathbf{a}) + \Delta t \mathbf{I} + \mathbf{e})$$

Table 1. Concrete message spaces for $N = 2^{16}$. Each ciphertext has N/n lanes, which together hold the indicated number of independent elements of the listed ring or field. The primes p_{128} , p_{256} , p_{512} , and p_{1024} are determined by [Proposition 7](#) for their respective rows and have 128, 256, 512, and 1024 bits, respectively. Both norms use the degree-less-than- n representatives in $\mathcal{S}_n$; the inverse norms are rounded to three decimal places.

n	t	$\ t\ _1$	$\ 1/t\ _1$	Ring / field	Elements per ciphertext
2^6	$X - 2$	3	1.000	$\mathbb{Z}_{2^{64}+1}$	1 024
2^6	$X^2 - 4$	5	0.333	$\mathbb{Z}_{2^{64}+1}$	2 048
2^{10}	$X - 2$	3	1.000	$\mathbb{Z}_{2^{1024}+1}$	64
2^{10}	$X^2 - 4$	5	0.333	$\mathbb{Z}_{2^{1024}+1}$	128
2^5	$X^{16} - 2$	3	0.600	$\mathbb{F}_{5^{16}}$	2 048
2^5	$X^8 - 2$	3	0.882	$\mathbb{F}_{17^4}$	4 096
2^6	$X^{32} - 6$	7	0.189	$\mathbb{F}_{37^{32}}$	1 024
2^{10}	$X^{128} - 2$	3	0.992	$\mathbb{F}_{257^8}$	1 024
2^6	$X^4 - X^3 + X + 4$	7	0.746	$\mathbb{F}_{p_{128}}$	1 024
2^7	$X^6 + X^2 + X + 4$	7	0.610	$\mathbb{F}_{p_{256}}$	512
2^8	$X^7 + X^6 + X + 4$	7	0.527	$\mathbb{F}_{p_{512}}$	256
2^9	$X^6 + X^5 + X^3 + 4$	7	0.627	$\mathbb{F}_{p_{1024}}$	128

for some $\mathbf{I} \in \mathcal{R}_n^{N/n}$ and $\mathbf{e} \in \mathcal{S}_n^{N/n}$ with $\|\mathbf{e}\|_\infty < \Delta/2$. In that case, we have $(\text{Dcd}_{\mathcal{A}_n} \circ \text{Dec}_{s,\ell})(\mathbf{ct}) = \mathbf{a}$, where the map $\text{Dcd}_{\mathcal{A}_n}$ is defined as follows:

$$\text{Dcd}_{\mathcal{A}_n} : \frac{\mathbb{Z}[X]}{\langle X^N + 1 \rangle} \rightarrow \mathcal{A}_n^{N/n}, \quad \text{Dcd}_{\mathcal{A}_n}(P) = \text{Proj}_n \left(\left\lfloor \frac{\Phi_{n \leftarrow N}(P)}{\Delta} \right\rfloor \right).$$

The maps underlying the scheme are summarized in [Figure 3](#).

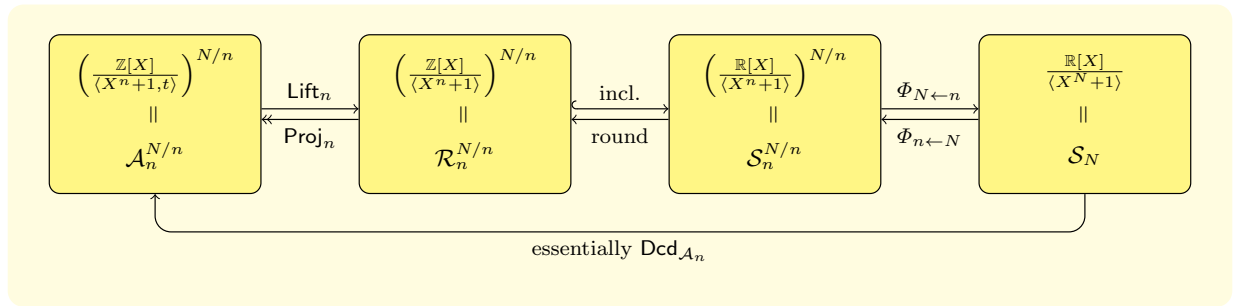


Fig. 3. High-level maps connecting the message space to the CKKS plaintext ring; the decoding map $\text{Dcd}_{\mathcal{A}_n}$ is the composition of the right-to-left maps, with an additional division by the scale Δ before coefficientwise rounding.

6.2 Correct Decryption

To deal more easily with studying the homomorphic operations below, we introduce some more general notation. Given $\mathbf{a} \in \mathcal{A}_n^{N/n}$ and two bounds B_I and B_e , we let $\mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ be the set of all ciphertexts $\mathbf{ct} \in \mathcal{R}_{N,q_\ell}^2$ for which there exist $\mathbf{I} \in \mathcal{R}_n^{N/n}$ and $\mathbf{e} \in \mathcal{S}_n^{N/n}$ satisfying $\|\mathbf{I}\|_\infty \leq B_I$ and $\|\mathbf{e}\|_\infty \leq B_e$ such that the following congruence holds:

$$\text{Dec}_{s,\ell}(\mathbf{ct}) \equiv_{q_\ell} \Phi_{N \leftarrow n}(\Delta \text{Lift}_n(\mathbf{a}) + \Delta t \mathbf{I} + \mathbf{e}).$$

Here and below, decryption congruences modulo q_ℓ are read in $\mathcal{R}_N$: the difference between the two sides lies in $q_\ell \mathcal{R}_N$, even though the message and error terms are expressed separately over the reals. We will say that the pair $(\mathbf{I}, \mathbf{e})$ *witnesses* that $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Since we have an equivalence modulo q_ℓ and not necessarily an equality, the decryption of $\mathbf{ct}$ may not yield $\mathbf{a}$. The following sufficient condition combines the lifting bound with bounds on the packing maps, established in [Appendix C](#). It ensures that decryption avoids wraparound modulo q_ℓ and that rounding recovers the message.

Proposition 8 (Correct decryption). *Let $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Then $(\text{Dcd}_{\mathcal{A}_n} \circ \text{Dec}_{s,\ell})(\mathbf{ct}) = \mathbf{a}$ if the following two inequalities are satisfied:*

$$\sqrt{n} \cdot \left(\Delta \|t\|_1 \left(\frac{1}{2} + B_I \right) + B_e \right) < \frac{q_\ell}{2}, \quad B_e < \frac{\Delta}{2}.$$

In what follows, we therefore only need to provide bounds B_I and B_e for each ciphertext; decryption is then correct if the bounds from [Proposition 8](#) are satisfied.

6.3 Fresh Encryption

We encrypt a message tuple by lifting each entry and packing the resulting polynomials into $\mathcal{S}_N$. After scaling by Δ and rounding coefficientwise, we apply standard CKKS encryption. This construction introduces no overflow; its error comes from rounding and encryption. For a fresh RLWE encryption error $e_{\text{enc}} \in \mathcal{R}_N$, we assume the following bound in the lane representation:

$$\|\Phi_{n \leftarrow N}(e_{\text{enc}})\|_\infty \leq B_{\text{enc},n}.$$

Proposition 9 (Fresh encryption). *Let $\mathbf{a} \in \mathcal{A}_n^{N/n}$ and $P = \Delta \Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}))$. Then a ciphertext $\mathbf{ct} \leftarrow \text{Enc}_s(\lfloor P \rfloor)$ satisfies $\mathbf{ct} \in \mathcal{C}_{s,L}(\mathbf{a}; B_I, B_e)$, where:*

$$B_I = 0, \quad B_e = B_{\text{enc},n} + \frac{N}{2\sqrt{n}}.$$

The proof is given in [Appendix C](#).

6.4 Homomorphic Operations

For every key-switching operation (including relinearization) and rescaling operation considered below, we assume that the introduced errors $e_{\text{ks}}, e_{\text{rs}} \in \mathcal{S}_N$ satisfy:

$$\|e_{\text{ks}}\|_\infty \leq B_{\text{ks}}, \quad \|\Phi_{n \leftarrow N}(e_{\text{ks}})\|_\infty \leq B_{\text{ks},n}, \quad \|\Phi_{n \leftarrow N}(e_{\text{rs}})\|_\infty \leq B_{\text{rs},n}.$$

The bounds $B_{\text{ks},n}$ and $B_{\text{rs},n}$ refer to the lane representation in $\mathcal{S}_n^{N/n}$, while B_{ks} refers to the native ring $\mathcal{S}_N$. The bounds apply uniformly over the levels and keys used below. All statements that follow are deterministic, conditional on these bounds holding for the operations considered. Following GBFV, we use worst-case bounds in our theoretical analysis and leave the derivation of tighter high-probability bounds as an open problem. For practical parameter selection, we suggest instead choosing these bounds heuristically from experimental measurements; such choices yield practical parameter sets, though without formal guarantees of low failure probabilities.

Addition changes the overflow by at most one beyond the input overflows. When one input is a plaintext, its encoding also contributes a rounding error.

Proposition 10 (Plaintext–ciphertext addition). *Consider a ciphertext $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and a plaintext $\mathbf{a}' \in \mathcal{A}_n^{N/n}$. Set $P = \Delta\Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}'))$ and $B'_e = \|\Phi_{n \leftarrow N}([P]_1)\|_\infty$. Then $\mathbf{ct}_{\text{add}} \in \mathcal{C}_{s,\ell}(\mathbf{a} + \mathbf{a}'; B_{I,\text{add}}, B_{e,\text{add}})$, where $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}([P], \mathbf{ct})$ and:*

$$B_{I,\text{add}} = B_I + 1, \quad B_{e,\text{add}} = B_e + B'_e.$$

For two ciphertext inputs, the existing approximation errors simply add.

Proposition 11 (Ciphertext–ciphertext addition). *Consider ciphertexts $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$. Then $\mathbf{ct}_{\text{add}} \in \mathcal{C}_{s,\ell}(\mathbf{a} + \mathbf{a}'; B_{I,\text{add}}, B_{e,\text{add}})$, where $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}(\mathbf{ct}, \mathbf{ct}')$ and:*

$$B_{I,\text{add}} = B_I + B'_I + 1, \quad B_{e,\text{add}} = B_e + B'_e.$$

Multiplication introduces products of representatives and errors. Their bounds depend on t through $\|t\|_1$, which controls the size of the lifted messages and overflow terms.

Proposition 12 (Plaintext–ciphertext multiplication). *For a level $\ell \in \llbracket 1, L \rrbracket$, consider a ciphertext $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and a plaintext $\mathbf{a}' \in \mathcal{A}_n^{N/n}$. Set $P = \Delta\Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}'))$ and $B'_e = \|\Phi_{n \leftarrow N}([P]_1)\|_\infty$. Then $\mathbf{ct}_{\text{mul}} \in \mathcal{C}_{s,\ell-1}(\mathbf{a} \cdot \mathbf{a}'; B_{I,\text{mul}}, B_{e,\text{mul}})$, where $\mathbf{ct}_{\text{mul}} \leftarrow \text{Rescale}(\text{Mul}([P], \mathbf{ct}))$ and:*

$$B_{I,\text{mul}} = \frac{n\|t\|_1}{2} \left(\frac{1}{2} + B_I \right) + \frac{1}{2},$$

$$B_{e,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + \frac{n\|t\|_1}{2} B_e + \frac{nB_e B'_e}{\Delta} + B_{\text{rs},n}.$$

With two ciphertext inputs, both overflows contribute to the product, and relinearization introduces an additional key-switching error.

Proposition 13 (Ciphertext–ciphertext multiplication). *For a level $\ell \in \llbracket 1, L \rrbracket$, consider ciphertexts $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$. Then $\mathbf{ct}_{\text{mul}} \in \mathcal{C}_{s,\ell-1}(\mathbf{a} \cdot \mathbf{a}'; B_{I,\text{mul}}, B_{e,\text{mul}})$, where $\mathbf{ct}_{\text{mul}} \leftarrow \text{Rescale}(\text{Mul}(\mathbf{ct}, \mathbf{ct}'))$ and:*

$$B_{I,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) \left(\frac{1}{2} + B'_I \right) + \frac{1}{2},$$

$$B_{e,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + n\|t\|_1 \left(\frac{1}{2} + B'_I \right) B_e + \frac{nB_e B'_e + B_{\text{ks},n}}{\Delta} + B_{\text{rs},n}.$$

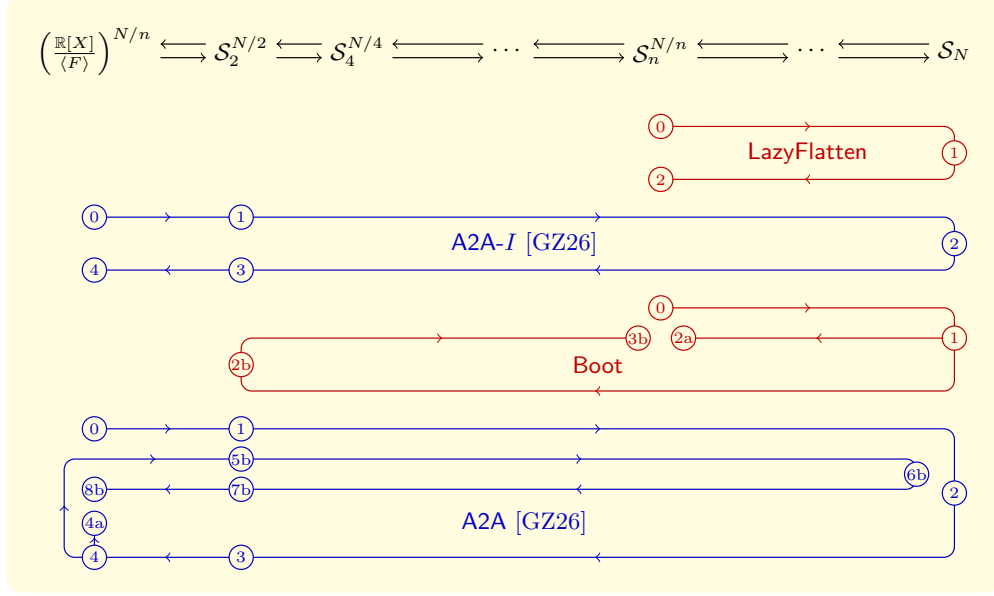


Fig. 4. Representation paths for the refreshing steps in our work (red) and in [GZ26] (blue). Here $F = X^n - X + 2$ is the defining polynomial of GZ’s arithmetic representation. Since $\mathcal{S}_2 \cong \mathbb{C}$, the space $\mathcal{S}_2^{N/2}$ is the complex-slot representation. Our two refreshing algorithms are **LazyFlatten**, which refreshes the overflow **I**, and **Boot**, which refreshes both the overflow **I** and the error **e**. In [GZ26, Section 5.1], **A2A-I** traverses the full representation path once. Arithmetic bootstrapping, **A2A**, traverses it twice, shown as a spiral: the outer loop is **A2A-I** (steps 0–4), and the inner loop extracts the error for **A2A-e** (steps 4–8b). At step 4, a branch retains the ciphertext at 4a. The extracted error is subtracted from the retained ciphertext: 3b from 2a in **Boot**, and 8b from 4a in **A2A**.

7 Ciphertext Refreshing

We provide two algorithms for refreshing a ciphertext $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$, similar to [GZ26]:

- **LazyFlatten** refreshes the overflow **I** and restores ciphertext levels, without reducing the error **e**.
- **Boot** refreshes both the overflow **I** and the error **e**, using the initial steps of lazy flattening.

Figure 4 shows the representation paths of these two algorithms. **Boot** branches in $\mathcal{S}_N$: one ciphertext returns directly to $\mathcal{S}_n^{N/n}$, while a second takes the longer path through $\mathcal{S}_2^{N/2}$ to extract the error. The two are then subtracted in $\mathcal{S}_n^{N/n}$. We do not consider isolated error refreshing, as omitting the overflow refreshing provides negligible performance improvements. **Boot** therefore refreshes both the overflow and the error. In the algorithms below, the gray annotations give, from left to right, the ring in which the encrypted plaintext is represented and the level of the ciphertext produced by each step.

7.1 Lazy Flattening

We use the polynomial $t \in \mathbb{Z}[X]$ from Section 4, of degree less than n and coprime to $X^n + 1$ in $\mathbb{Q}[X]$. Lazy flattening restores ciphertext levels while replacing the overflow vector **I** by a new vector $\tilde{\mathbf{I}}$ with a small, predictable bound $\tilde{B}_I$. It avoids the cost of removing the overflow entirely. Following [GZ26], the idea is simply to reduce the ciphertext modulo q_0 , before raising the modulus

back up; we perform the modulus raising under sparse-secret encapsulation [BTPH22], so that the new overflow vector can be bounded in terms of the ephemeral secret's Hamming weight h' .

Specifically, we fix an ephemeral ternary secret key $s' \in \mathcal{R}_N$ of small Hamming weight h' . We assume that key-switching keys are available from s to s' at level $\ell = 0$ and from s' to s at level $\ell = L$. The security considerations for this construction are discussed in [BTPH22]. We also precompute encodings of the following matrices:

$$\mathbf{V}_{N \leftarrow n} = \frac{q_0}{\Delta} \cdot \mathbf{U}_{N \leftarrow n} \cdot \mathbf{D}_t^{-1}, \quad \mathbf{V}_{n \leftarrow N} = \frac{\Delta}{q_0} \cdot \mathbf{D}_t \cdot \mathbf{U}_{n \leftarrow N},$$

where $\mathbf{D}_t = \mathbf{I}_{N/n} \otimes \text{diag}(\text{eval}_n(t)) \in \mathbb{C}^{(N/2) \times (N/2)}$ represents componentwise multiplication by t in $\mathcal{S}_n^{N/n}$. The matrix $\mathbf{V}_{N \leftarrow n}$ divides the elements in $\mathcal{S}_n$ by t , moves the resulting coefficients to $\mathcal{S}_N$, and changes the scaling factor from Δ to q_0 ; the inverse matrix $\mathbf{V}_{n \leftarrow N}$ performs the reverse operations. Both $\mathbf{V}_{N \leftarrow n}$ and $\mathbf{V}_{n \leftarrow N}$ admit the same efficient evaluation methods as $\mathbf{U}_{N \leftarrow n}$ and $\mathbf{U}_{n \leftarrow N}$. We denote the multiplicative depth chosen for the homomorphic evaluation of $\mathbf{V}_{N \leftarrow n}$ and $\mathbf{V}_{n \leftarrow N}$ by $\ell_{N \leftarrow n}, \ell_{n \leftarrow N} \in \llbracket 1, \log_2(N/n) \rrbracket$, respectively. Both refreshing algorithms below take their input at level $\ell_{N \leftarrow n}$. We assume that, for the inputs considered here, homomorphic multiplication by $\mathbf{V}_{N \leftarrow n}$ introduces an additional error $e_{N \leftarrow n} \in \mathcal{S}_N$ with $\|e_{N \leftarrow n}\|_\infty \leq B_{N \leftarrow n}$, and that homomorphic multiplication by $\mathbf{V}_{n \leftarrow N}$ introduces an additional error $\mathbf{e}_{n \leftarrow N} \in \mathcal{S}_n^{N/n}$ (here and below, this is the image under $\Phi_{n \leftarrow N}$ of the actual error introduced in $\mathcal{S}_N$) in the output lane representation at scale Δ , with $\|\mathbf{e}_{n \leftarrow N}\|_\infty \leq B_{n \leftarrow N}$. For related error estimates, see for instance [KPK⁺22].

Algorithm 1: LazyFlatten

Input: $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ with $\ell = \ell_{N \leftarrow n}$.

Output: $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s,\tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$ with $\tilde{\ell} = L - \ell_{n \leftarrow N}$.

1: $\mathbf{ct} \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{N \leftarrow n}), \mathbf{ct}_{\text{in}})$	$\mathcal{S}_N$	0
2: $\mathbf{ct} \leftarrow \text{KeySwitch}_{s' \leftarrow s}(\mathbf{ct})$	$\mathcal{S}_N$	0
3: $\mathbf{ct} \leftarrow \text{ModRaise}_{L \leftarrow 0}(\mathbf{ct})$	$\mathcal{S}_N$	L
4: $\mathbf{ct} \leftarrow \text{KeySwitch}_{s \leftarrow s'}(\mathbf{ct})$	$\mathcal{S}_N$	L
5: $\mathbf{ct}_{\text{out}} \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{n \leftarrow N}), \mathbf{ct})$	$\mathcal{S}_n^{N/n}$	$\tilde{\ell}$
6: return $\mathbf{ct}_{\text{out}}$		

The following result establishes the correctness of lazy flattening; it is proven in [Appendix D](#).

Proposition 14 (Correctness of lazy flattening). *Set $\ell = \ell_{N \leftarrow n}$ and $\tilde{\ell} = L - \ell_{n \leftarrow N}$, and suppose $\tilde{\ell} \geq 0$. Consider $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Then [Algorithm 1](#) returns $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s,\tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$, where:*

$$\tilde{B}_I = \frac{h'}{2} + 1 + \frac{\|\frac{1}{t}\|_1 B_e}{\Delta} + \frac{B_{N \leftarrow n} + B_{\text{ks}}}{q_0},$$

$$\tilde{B}_e = B_e + \frac{\Delta \|t\|_1}{q_0} (B_{N \leftarrow n} + 2B_{\text{ks}}) + B_{n \leftarrow N}.$$

For typical parameter sets and practical choices of t , we may simplify the bound to $\tilde{B}_I = h'/2 + 1$, since the remaining terms in the bound for $\tilde{B}_I$ in [Proposition 14](#) are much smaller than $1/2$ and the overflow has integer coefficients.

7.2 Arithmetic Bootstrapping

Algorithm 2 refreshes both the overflow and the approximation error. It begins with the initial steps of lazy flattening, then multiplies the ciphertext by $t(X^{N/n})$ while it is still represented in $\mathcal{S}_N$. The message and overflow contributions are then multiples of q_0 . We retain this ciphertext and reduce a copy modulo q_0 to isolate the approximation error. This error branch undergoes a second modulus raising, followed by a conversion to complex slots. In this representation, **EvalMod** removes the fresh integer overflow introduced by the second modulus raising, approximately recovering the isolated error. Finally, we return both branches to $\mathcal{S}_n^{N/n}$ and subtract the error branch from the retained ciphertext.

Here, **EvalMod** homomorphically approximates reduction modulo one on both the real and imaginary parts of each slot. We require it to map each component $x + e$, where $x \in \mathbb{Z}$ with $|x| \leq h'/2 + 1$ and $|e| \leq \varepsilon_{\text{em}} < 1/2$, to $e + e_{\text{em}}$ with $|e_{\text{em}}| \leq B_{\text{em}}/\Delta$. Thus, B_{em} bounds the additional error before division by the output scale Δ , including approximation and homomorphic evaluation errors. We assume that **EvalMod** consumes ℓ_{em} levels.

We set:

$$\mathbf{W}_{2 \leftarrow N} := \frac{\Delta}{q_0} \cdot \mathbf{U}_{2 \leftarrow N}, \quad \mathbf{W}_{n \leftarrow N} := \frac{\Delta}{q_0} \cdot \mathbf{U}_{n \leftarrow N}.$$

The level costs of $\mathbf{V}_{N \leftarrow n}$, $\mathbf{W}_{2 \leftarrow N}$, $\mathbf{U}_{n \leftarrow 2}$, and $\mathbf{W}_{n \leftarrow N}$ are denoted by $\ell_{N \leftarrow n}$, $\ell_{2 \leftarrow N}$, $\ell_{n \leftarrow 2}$, and $\ell_{n \leftarrow N}$, respectively. In particular, $\ell_{n \leftarrow 2} = 0$ when $n = 2$. When the additional errors introduced by these transformations in their respective output representations are $e_{N \leftarrow n} \in \mathcal{S}_N$, $\mathbf{e}_{2 \leftarrow N} \in \mathcal{S}_2^{N/2}$, and $\mathbf{e}_{n \leftarrow 2}, \mathbf{e}_{n \leftarrow N} \in \mathcal{S}_n^{N/n}$, we assume that their infinity norms are bounded by $B_{N \leftarrow n}$, $B_{2 \leftarrow N}$, $B_{n \leftarrow 2}$, and $B_{n \leftarrow N}$, respectively. These errors are measured before division by the output scale: q_0 for $\mathbf{V}_{N \leftarrow n}$ and Δ for the other three transformations. We assume that key-switching keys are available from s to s' at level 0 and from s' to s at levels $\tilde{\ell} + \ell_{n \leftarrow N}$ and L , where $\tilde{\ell} = L - \ell_{2 \leftarrow N} - \ell_{\text{em}} - \ell_{n \leftarrow 2}$.

Algorithm 2: Boot

Input: $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s, \ell}(\mathbf{a}; B_I, B_e)$ with $\ell = \ell_{N \leftarrow n}$.

Output: $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s, \tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$ with $\tilde{\ell} = L - \ell_{2 \leftarrow N} - \ell_{\text{em}} - \ell_{n \leftarrow 2}$.

1: $\mathbf{ct} \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{N \leftarrow n}), \mathbf{ct}_{\text{in}})$	$\mathcal{S}_N$	0
2: $\mathbf{ct} \leftarrow \text{KeySwitch}_{s' \leftarrow s}(\mathbf{ct})$	$\mathcal{S}_N$	0
3: $\mathbf{ct} \leftarrow \text{ModRaise}_{\tilde{\ell} + \ell_{n \leftarrow N} \leftarrow 0}(\mathbf{ct})$	$\mathcal{S}_N$	$\tilde{\ell} + \ell_{n \leftarrow N}$
4: $\mathbf{ct} \leftarrow t(X^{N/n}) \cdot \mathbf{ct}$	$\mathcal{S}_N$	$\tilde{\ell} + \ell_{n \leftarrow N}$
5: $\mathbf{ct}' \leftarrow [\mathbf{ct}]_{q_0}$	$\mathcal{S}_N$	0
6: $\mathbf{ct}' \leftarrow \text{ModRaise}_{L \leftarrow 0}(\mathbf{ct}')$	$\mathcal{S}_N$	L
7: $\mathbf{ct}' \leftarrow \text{KeySwitch}_{s \leftarrow s'}(\mathbf{ct}')$	$\mathcal{S}_N$	L
8: $\mathbf{ct}' \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{2 \leftarrow N}), \mathbf{ct}')$	$\mathcal{S}_2^{N/2}$	$L - \ell_{2 \leftarrow N}$
9: $\mathbf{ct}' \leftarrow \text{EvalMod}(\mathbf{ct}')$	$\mathcal{S}_2^{N/2}$	$L - \ell_{2 \leftarrow N} - \ell_{\text{em}}$
10: $\mathbf{ct}' \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{U}_{n \leftarrow 2}), \mathbf{ct}')$	$\mathcal{S}_n^{N/n}$	$\tilde{\ell}$
11: $\mathbf{ct} \leftarrow \text{KeySwitch}_{s \leftarrow s'}(\mathbf{ct})$	$\mathcal{S}_N$	$\tilde{\ell} + \ell_{n \leftarrow N}$
12: $\mathbf{ct} \leftarrow \text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{n \leftarrow N}), \mathbf{ct})$	$\mathcal{S}_n^{N/n}$	$\tilde{\ell}$
13: $\mathbf{ct}_{\text{out}} \leftarrow [\mathbf{ct} - \mathbf{ct}']_{q_{\tilde{\ell}}}$	$\mathcal{S}_n^{N/n}$	$\tilde{\ell}$
14: return $\mathbf{ct}_{\text{out}}$		

The following result establishes the correctness of arithmetic bootstrapping; it is proven in [Appendix E](#).

Proposition 15 (Correctness of arithmetic bootstrapping). Set $\ell = \ell_{N \leftarrow n}$ and $\tilde{\ell} = L - \ell_{2 \leftarrow N} - \ell_{\text{em}} - \ell_{n \leftarrow 2}$, and suppose $\tilde{\ell} \geq 0$ and $L \geq \tilde{\ell} + \ell_{n \leftarrow N}$. Consider $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Assume the following condition:

$$\frac{B_e + B_{2 \leftarrow N}}{\Delta} + \frac{\|t\|_1(B_{N \leftarrow n} + B_{\text{ks}}) + B_{\text{ks}}}{q_0} \leq \varepsilon_{\text{em}}. \quad (1)$$

Then [Algorithm 2](#) returns $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s,\tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$, where:

$$\begin{aligned} \tilde{B}_I &= \frac{h'}{2} + 1 + \frac{\|\frac{1}{t}\|_1 B_e}{\Delta} + \frac{B_{N \leftarrow n} + B_{\text{ks}}}{q_0}, \\ \tilde{B}_e &= B_{n \leftarrow N} + B_{2 \leftarrow N} + B_{\text{em}} + B_{n \leftarrow 2} + \frac{2\Delta}{q_0} B_{\text{ks}}. \end{aligned}$$

The overflow bound is the same as for lazy flattening, while the error carried by the retained ciphertext is canceled algebraically. The remaining approximation error still depends on the chosen input domain through B_{em} .

8 Experimental Results

We evaluate our framework on modular integer rings and selected finite-field message spaces. For modular integer arithmetic, we compare refreshing and multiplication costs with GZ. We also test whether LazyFlatten and Boot take longer when the lanes store finite-field elements instead of modular integers. We compare the two settings at the same lane degree, with the same scale, modulus chain, and transformation depths. We also profile the main steps of arithmetic bootstrapping and study the normalized output error.

8.1 Implementation and Setup

All measurements performed for this work were taken on an Apple M4 Pro with 14 cores and 24 GB of memory, running macOS 15.7. Each benchmark uses a single thread on an otherwise idle machine. Our implementation uses Go 1.26 and Lattigo v6.1 [Lat24]. We discard warm-up timings before collecting the reported measurements. The warm-up runs populate the cache of encoded transformations for the input scales encountered during evaluation. Published timings and extrapolations are identified separately.

For a ciphertext at scale S , we measure the normalized coefficient error $\|\mathbf{e}\|_\infty/S$ in the lane representation, taking the maximum over all coefficients in all lanes. We report either this quantity or its base-two logarithm. The output of refreshing has scale $S = \Delta$, so its normalized error is $\|\tilde{\mathbf{e}}\|_\infty/\Delta$. We use the corresponding coefficient-error measure for GZ.

The framework allows general plaintext moduli t satisfying the assumptions of [Section 6](#). The implementation used for the measurements in this section is specialized to $t = X^r - c$. In particular, the four non-binomial prime-field examples in [Table 1](#) are not included in these benchmarks. For the binomial cases, we implement in Lattigo: lifting and packing, homomorphic multiplication in $\mathcal{A}_n^{N/n}$, LazyFlatten ([Algorithm 1](#)), and Boot ([Algorithm 2](#)). Messages are represented as polynomials in $\mathcal{A}_n \cong \mathbb{Z}_M[X]/\langle X^r - c \rangle$, where $M = c^{n/r} + 1$. This includes modular integer rings, extension fields, and products of such rings. In the benchmarks, each multiplication followed by Boot is checked by decryption against an independently computed product in this quotient ring.

The linear transformations use the factorizations of [Section 3.5](#), evaluated with baby-step giant-step and hoisted rotations [\[HS18\]](#). Their diagonal factors are computed with 256-bit precision. [Appendix F](#) describes the implementation’s scale adjustments, level choices, and key-switching schedule.

8.2 Parameters and Message Spaces

The timing experiments use RLWE dimension $N = 2^{16}$ and N/n lanes per ciphertext; we vary N separately in the precision study of [Section F.4](#). Our scale is $\Delta = 2^{60}$, with a ciphertext-modulus chain of 56- and 60-bit primes. The chain depends on the chosen depths of the partial transformations. These choices apply to both the modular integer and finite-field experiments.

For the modular integer benchmarks, we choose $t = X - 2$, so each lane carries an element of $\mathcal{A}_n \cong \mathbb{Z}_{2^n+1}$. The GZ baseline carries an integer modulo 2^n in each polynomial representative, with the same number N/n of messages per ciphertext. For GZ, we retain the parameters of [\[GZ26\]](#): $\Delta = 2^{43}$ and a chain of 21 primes of 43 bits. [Table 2](#) summarizes this comparison. The finite-field experiments use the binomials listed in [Table 4](#); [Section 8.4](#) describes their message spaces and paired baselines.

In the RNS implementation, we retain q_ℓ for the full ciphertext modulus at level ℓ and write $p_\ell = q_\ell/q_{\ell-1}$ for the prime removed by rescaling, for $\ell \geq 1$. We denote the product of the auxiliary key-switching primes by P_{aux} . The ephemeral secret s' has Hamming weight $h' = 32$ in both schemes. The key-switching key to s' uses only the modulus $q_0 p_{\text{aux},0}$, where $p_{\text{aux},0}$ is one auxiliary prime.

Table 2. Parameters for the modular integer comparison at $N = 2^{16}$, with $t = X - 2$ for our scheme. The quantity $\log_2(q_L P_{\text{aux}})$ gives the bit length of the largest modulus used for an evaluation key.

Parameter	Ours	GZ
Message space	$\mathbb{Z}_{2^n+1}^{N/n}$	$\mathbb{Z}_{2^n}^{N/n}$
Scale Δ	2^{60}	2^{43}
Ciphertext-modulus chain	56- and 60-bit primes	21 primes of 43 bits
Auxiliary modulus P_{aux}	3 primes of 61 bits	7 primes of 50 bits
$\log_2(q_L P_{\text{aux}})$	1119–1287 (Boot)	1253
Main secret s	Uniform ternary	Sparse ternary, $h = 192$
Ephemeral secret s'	$h' = 32$	$h' = 32$
$\log_2(q_0 p_{\text{aux},0})$	121	93

8.3 Refreshing over Modular Integer Rings

Porting GZ into Lattigo. For the modular integer comparison, we use a Lattigo port of GZ and the original C++ prototype accompanying [\[GZ26\]](#). The original prototype is built on OpenFHE [\[ABAB⁺22\]](#). The timings in [\[GZ26\]](#) use Intel HEXL on an x86 machine. To reduce hardware and library differences

in the comparison, our port uses the same `Go` and `Lattigo` versions as our implementation. The port includes the triangle encoding in $\mathbb{Z}[X]/\langle X^n - X + 2 \rangle$, the blockwise transformations between groups of $n/2$ complex slots and polynomial representatives, `A2A-I` for overflow refreshing, and `A2A-e` for error refreshing. The arithmetic bootstrapping procedure `A2A` consists of `A2A-I` followed by `A2A-e`. We follow the optimizations of [GZZ26]: the decomposition into shifted diagonals, baby-step giant-step evaluation with hoisted rotations, three levels for `CoeffToSlot` and two for `SlotToCoeff`, sparse-secret encapsulation for modulus raising, and a degree-32 Chebyshev approximation followed by three double-angle steps for `EvalMod`.

This port provides running times on the same machine and in the same library, as well as the multiplication counts needed for Table 5. Two rounds of measurements interleaved the two schemes and reversed their order in the second round; the results agreed within 1% for almost every configuration. The original prototype cannot use Intel HEXL on the Apple M4 Pro. Without HEXL, it takes 12.2 and 60.4 seconds at $n = 8$ for `A2A-I` and `A2A`, respectively. The corresponding published times are 4.3 and 10.1 seconds, while our port takes 3.7 and 9.3 seconds, within 15% of the published values.

The range of same-machine comparisons is limited by memory. For hoisted baby-step giant-step evaluation, `Lattigo` stores the encoded diagonals in the extended modulus used for key switching. Under the machine’s 24 GB memory limit, our port supports `A2A-I` up to $n = 128$ and `A2A` up to $n = 64$. The original prototype uses 16–18 GB at $n = 256$.

We use the measured times of our `Lattigo` port within these ranges. For larger n up to 256, we instead use the timings reported in [GZZ26] for the original `OpenFHE` implementation with Intel HEXL on x86; these comparisons therefore involve different hardware. For $n > 256$, which is outside the published measurement range, we estimate GZ’s arithmetic bootstrapping time by extrapolating the published timings. Table 3 distinguishes the port measurements, published timings, and extrapolated estimates; we describe the extrapolation below.

Refreshing Times. Table 3 compares the refreshing procedures of both schemes, and Figure 2 shows their arithmetic bootstrapping times as the lane degree n varies. For $n = 8$ to $n = 128$, `LazyFlatten` is $4.2\times$ – $16\times$ faster than GZ’s `A2A-I` in the same library. For $n = 8$ to $n = 64$, `Boot` is $2.2\times$ – $3.8\times$ faster than GZ’s `A2A`. As n grows, the running time of `Boot` remains nearly constant, while that of `LazyFlatten` decreases. The profiling results below explain this behavior. In GZ, refreshing becomes more expensive overall, as its blockwise transformations use dense $(n/2) \times (n/2)$ matrices.

The port reaches its memory limit at $n = 128$ for `A2A-I` and at $n = 64$ for `A2A`, which stores the encoded diagonals of both `A2A-I` and `A2A-e` simultaneously. The original prototype includes a precomputed root table for $X^n - X + 2$ only up to $n = 256$. We extended this table using roots computed with 128-bit precision, reproducing the existing entries digit for digit. The extended prototype produced correct results at $n = 512$ and $n = 1024$, but exceeded the available memory and required sustained paging. We therefore exclude those runs from the timing comparison.

For larger n , we extrapolate the published x86/HEXL timings using a least-squares fit against $\sqrt{n}$:¹ the estimates are 30.3, 52.6, and 97.2 seconds at $n = 2^{10}$, 2^{12} , and 2^{14} , respectively. Our measured times at these degrees are 3.84, 3.52, and 3.69 seconds, giving estimated speedups of $7.9\times$,

¹ Fitting $n = 8$ to 256 gives $\hat{T}_{\text{GZ}}(n) = 7.96 + 0.70\sqrt{n}$ seconds, with a maximum relative error of 3.6% on the fitted points. Baby-step giant-step evaluation gives a rotation count proportional to $\sqrt{n}$, but the number of plaintext–ciphertext multiplications grows linearly in n . The fit is therefore an empirical model, not an asymptotic bound.

Table 3. Refreshing times in seconds at $N = 2^{16}$ and $t = X - 2$. The GZ entries give the times of our **Lattigo** port; parentheses contain the published **OpenFHE** times on x86 with Intel **HEXL** [GZ26]. A dash in a GZ timing column indicates that the port exceeds 24 GB of memory. Arithmetic bootstrapping speedups beyond $n = 2^6$ use the published times; stars mark extrapolated times and the resulting speedups. Times measured for this work are medians of ten repetitions, excluding warm-up runs.

n	N/n	Overflow refreshing			Overflow and error refreshing		
		LazyFlatten	A2A-I	Speedup	Boot	A2A	Speedup
2^3	8 192	0.89	3.73 (4.3)	$4.17\times$	4.31	9.30 (10.1)	$2.16\times$
2^4	4 096	0.75	3.80 (4.7)	$5.10\times$	4.17	9.38 (10.9)	$2.25\times$
2^5	2 048	0.64	4.47 (5.3)	$7.00\times$	4.13	10.41 (11.8)	$2.52\times$
2^6	1 024	0.42	6.37 (6.4)	$15.1\times$	3.91	14.92 (13.5)	$3.82\times$
2^7	512	0.39	6.24 (7.7)	$15.8\times$	3.89	– (15.3)	$3.9\times$
2^8	256	0.35	– (10.2)	–	3.92	– (19.5)	$5.0\times$
2^{10}	64	0.21	–	–	3.84	– (30.3*)	$7.9\times^*$
2^{12}	16	0.12	–	–	3.52	– (52.6*)	$14.9\times^*$
2^{14}	4	0.07	–	–	3.69	– (97.2*)	$26.4\times^*$

$15\times$, and $26\times$. Fitting the port’s timings instead would increase the estimates by approximately 50% across these degrees. The comparisons beyond the measured range remain dependent on the extrapolation: at $n = 2^{14}$, the degree is 64 times the largest degree used in the fit.

8.4 Refreshing over Finite Fields

We also benchmark the binomial finite-field message spaces of Table 4, using the same implementation with different values of (n, r, c) . The first two choices give $\mathcal{A}_n \cong \mathbb{F}_{5^{16}}$ and $\mathcal{A}_n \cong \mathbb{F}_{37^{32}}$, respectively. The third gives $\mathcal{A}_n \cong \mathbb{F}_{257^{16}}$, so each of its $N/n = 64$ lanes contains sixteen field elements. The benchmarks multiply encrypted messages and apply **Boot**, checking the decrypted products against plaintext arithmetic; they also measure **LazyFlatten** separately.

These experiments assess the cost of changing the message space at a fixed lane degree. The Fourier stages depend only on n ; the diagonal factors $\mathbf{D}_t$ and $\mathbf{D}_t^{-1}$ account for the choice of t . We compare each finite-field configuration $t = X^r - c$ with a modular-integer baseline $t = X - c$, whose message ring is $\mathbb{Z}_{c^n+1}$. The two configurations use the same n and c , scale, modulus chain, and transformation depths. Each pair is measured in the same process, so only r changes. The measured refreshing times closely match those of the corresponding baselines: the selected finite-field message spaces add little cost to **LazyFlatten** and **Boot**.

8.5 Amortized Cost of Multiplication

We return to modular integer arithmetic with $t = X - 2$ to measure the cost of sustained computation. Several multiplications can be evaluated between successive arithmetic bootstrapping calls, provided the accumulated overflow and error remain within the supported ranges. Since **LazyFlatten** controls the overflow and restores ciphertext levels at a lower cost than **Boot**, we alternate multiplication with lazy flattening and apply **Boot** when the error approaches the supported input bound. Table 5 gives the measured number k of multiplications between successive arithmetic bootstrapping calls

Table 4. Refreshing times in seconds for three extension-field message spaces at $N = 2^{16}$. The finite-field columns use the polynomial t shown in the row. The modular-integer columns use $t = X - c$ with the same n and c : in particular, $t = X - 6$ for the $\mathbb{F}_{3732}$ row. In the last row, each lane holds sixteen $\mathbb{F}_{2578}$ elements.

Field	n	t	Finite-field messages		Modular-integer baseline	
			LazyFlatten	Boot	LazyFlatten	Boot
$\mathbb{F}_{516}$	2^5	$X^{16} - 2$	0.64	4.12	0.64	4.12
$\mathbb{F}_{3732}$	2^6	$X^{32} - 6$	0.42	3.93	0.44	3.95
$\mathbb{F}_{2578}$	2^{10}	$X^{128} - 2$	0.21	3.87	0.21	3.88

and the resulting amortized cost per multiplication. Writing T_{mul} , T_{lazy} , and T_{boot} for the running times of multiplication, **LazyFlatten**, and **Boot**, respectively, our amortized cost is:

$$T_{\text{mul}} + T_{\text{lazy}} + \frac{T_{\text{boot}}}{k}.$$

For **GZ**, each multiplication is followed by **A2A-I**, and the cycle ends with **A2A-e**. Using the corresponding running times, its amortized cost is:

$$T_{\text{mul}} + T_{\text{A2A-I}} + \frac{T_{\text{A2A-e}}}{k}.$$

Table 5. Multiplications k per arithmetic bootstrapping cycle and estimated amortized time per multiplication at $N = 2^{16}$ and $t = X - 2$. Times are computed from the measured operation times using the formulas above; the per-lane time divides the per-ciphertext time by N/n . A count k is accepted only if all fifty tested cycles decrypt correctly and return to the observed error floor. Each multiplication is followed by **LazyFlatten** or **A2A-I**; each cycle ends with **Boot** or **A2A-e**, respectively. The **GZ** port exceeds the available memory at $n = 2^{12}$.

n	Ours			GZ			Speedup
	k	Seconds per ciphertext	Milliseconds per lane	k	Seconds per ciphertext	Milliseconds per lane	
2^3	4	1.99	0.24	2	6.62	0.81	3.3×
2^6	3	1.74	1.70	1	15.0	14.6	8.6×
2^{12}	2	1.89	118	–	–	–	–

Direct measurements of our evaluation loop give 1.99, 1.76, and 1.95 seconds per multiplication per ciphertext at $n = 2^3$, 2^6 , and 2^{12} , respectively, within 4% of the estimates in [Table 5](#). These multiplication counts are empirical. Three observations help explain the difference between the schemes. First, multiplication consumes one level in our scheme and two in **GZ**, where an additional multiplication by the plaintext modulus polynomial restores the arithmetic representation. Second, the normalized error ranges from approximately $2^{-35.4}$ after **Boot** to an admissible input error of roughly 2^{-16} , a budget of about 19 bits. The corresponding range for **GZ** is $2^{-23.4}$ to roughly 2^{-8} , or about 15 bits, using Theorem 3 of [\[GZ26\]](#). Third, at $n = 8$, the measured error growth per multiplication is about 5 bits in our scheme and 8 bits in **GZ**. Both measurements use repeated

squaring, as in the benchmark of [GZ26]. The ratios $19/5 = 3.8$ and $15/8 \approx 1.9$ are consistent with the measured counts of four and two multiplications.

One may instead apply lazy flattening every j multiplications, using $j - 1$ additional primes in the modulus chain. For $j \in \{1, 2, 4\}$, this gave no performance improvement in our experiments. The larger overflow increased the error introduced by refreshing and reduced the number k of multiplications between arithmetic bootstrapping calls. At the tested parameters, this reduction outweighed the saved lazy-flattening calls.

8.6 Profiled Benchmarks

Table 6 gives the running times of the main steps of **Boot**, using the transformation notation of Section 7. The coefficient-to-slot transformation $\mathbf{W}_{2 \leftarrow N}$ and **EvalMod** account for approximately 75%–82% of the total; their transformation size and evaluation parameters are fixed as n varies. The partial transformations account for most of the remaining variation. The forward and retained-branch transformations span $\log_2(N/n)$ Fourier stages, while $\mathbf{U}_{n \leftarrow 2}$ spans $\log_2(n/2)$ stages. These two stage counts sum to $\log_2(N/2)$, so shortening the lane-to-coefficient path lengthens the return path from slots. This explains the nearly constant running time of **Boot**, with the smallest measured times near $n = 2^{12}$. **LazyFlatten** uses only the transformations between lanes and $\mathcal{S}_N$, and its running time decreases from 0.89 to 0.07 seconds in Table 3.

At $n = 8$ and $n = 2^{12}$, we also vary the linear plaintext modulus $t = X - \beta$ over $\beta \in \{2, 8, 26, 64\}$. The total time changes by less than 2%, and the normalized output error remains between $2^{-35.5}$ and $2^{-34.8}$. In these tests, the error branch uses the same **EvalMod** parameters for every value of β .

Table 6. Running times of the main steps of **Boot** in seconds, at $N = 2^{16}$ and $t = X - 2$. Each step time is the median of ten repetitions after two warm-up runs; the totals sum the median step times. The modulus-raising row includes both calls and their surrounding key switches. The totals also include multiplication by $t(X^{N/n})$ and final subtraction. Scale adjustments are folded into the transformations as described in Appendix F. The totals differ from Table 3 by less than 1% at $n = 2^3$ and $n = 2^{12}$.

Operation	$n = 2^3$	$n = 2^{12}$
LinTrans($\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{N \leftarrow n}), \cdot$)	0.31	0.03
ModRaise and key switching	0.13	0.12
LinTrans($\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{2 \leftarrow N}), \cdot$)	1.82	1.65
EvalMod	1.42	1.25
LinTrans($\text{Ecd}_{\mathbb{C}}(\mathbf{U}_{n \leftarrow 2}), \cdot$)	0.07	0.43
LinTrans($\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{n \leftarrow N}), \cdot$)	0.56	0.06
Total	4.30	3.55
Share of $\mathbf{W}_{2 \leftarrow N}$ and EvalMod	75%	82%

8.7 Precision

Across the tested input errors, input scales, and repeated refreshing calls, the normalized output error of **Boot** remains close to 2^{-35} at $N = 2^{16}$. With a sparse main secret of Hamming weight

$h = 192$, it is approximately 2^{-37} . The corresponding normalized error is approximately 2^{-23} for our GZ port and $2^{-24.4}$ in [GZ26, Table 3].

A separate diagnostic experiment in Section F.4 examines the dependence of the output error on N and n , including smaller dimensions that are not secure with the fixed modulus chain.

9 Ongoing Work

We are developing arithmetic-to-digit and digit-to-arithmetic conversion in our setting of CKKS-native polynomial lanes, for inclusion in a future version of this paper, similar to the conversions in GZ [GZ26]. We also plan a broader experimental comparison with recent and concurrent work. Beyond GZ [GZ26], the comparison will include its extensions to large prime moduli by Wei *et al.* [WMGZ26] and arbitrary moduli by Liu *et al.* [LMWW26].

Acknowledgments. ChatGPT was used for proofreading and improving the clarity of the text. Claude Fable and Opus were used for assisting and improving the implementation. Tim Seuré acknowledges the support of the Luxembourgish “Fonds National de la Recherche” (FNR) through an Individual Grant (reference number: 17936291).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

- ABAB⁺22. Ahmad Al Badawi, Andreea Alexandru, Jack Bates, Flavio Bergamaschi, David Bruce Cousins, Saroja Erabelli, Nicholas Genise, Shai Halevi, Hamish Hunt, Andrey Kim, Yongwoo Lee, Zeyu Liu, Daniele Micciancio, Carlo Pascoe, Yuriy Polyakov, Ian Quah, Saraswathy R.V., Kurt Rohloff, Jonathan Saylor, Dmitriy Sponitsky, Matthew Triplett, Vinod Vaikuntanathan, and Vincent Zucca. **OpenFHE**: Open-Source Fully Homomorphic Encryption Library. Cryptology ePrint Archive, Paper 2022/915, 2022. <https://eprint.iacr.org/2022/915>.
- AKP25. Andreea Alexandru, Andrey Kim, and Yuriy Polyakov. General Functional Bootstrapping Using CKKS. In *Advances in Cryptology – CRYPTO 2025*. Springer, 2025. doi:10.1007/978-3-032-01881-6_10.
- AKPZ26. Andreea Alexandru, Andrey Kim, Yuriy Polyakov, and Hongren Zheng. Sparse Hermite Interpolation Method for Discrete-CKKS Functional Bootstrapping. Cryptology ePrint Archive, Paper 2026/1026, 2026. <https://eprint.iacr.org/2026/1026>.
- BCKS24. Youngjin Bae, Jung Hee Cheon, Jaehyung Kim, and Damien Stehlé. Bootstrapping Bits with CKKS. In *Advances in Cryptology – EUROCRYPT 2024*. Springer, 2024. doi:10.1007/978-3-031-58723-8_4.
- BGV12. Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (Leveled) Fully Homomorphic Encryption without Bootstrapping. In *ITCS 2012: Innovations in Theoretical Computer Science*. ACM, 2012. doi:10.1145/2090236.2090262.
- BK25. Dan Boneh and Jaehyung Kim. Homomorphic Encryption for Large Integers from Nested Residue Number Systems. In *Advances in Cryptology – CRYPTO 2025*. Springer, 2025. doi:10.1007/978-3-032-01881-6_11.
- BKSS25. Youngjin Bae, Jaehyung Kim, Damien Stehlé, and Elias Suvanto. Bootstrapping Small Integers with CKKS. In *Advances in Cryptology – ASIACRYPT 2024*. Springer, 2025. doi:10.1007/978-981-96-0875-1_11.
- Bra12. Zvika Brakerski. Fully Homomorphic Encryption without Modulus Switching from Classical GapSVP. In *Advances in Cryptology – CRYPTO 2012*. Springer, 2012. doi:10.1007/978-3-642-32009-5_50.
- BTPH22. Jean-Philippe Bossuat, Juan Troncoso-Pastoriza, and Jean-Pierre Hubaux. Bootstrapping for Approximate Homomorphic Encryption with Negligible Failure-Probability by Using Sparse-Secret Encapsulation. In *Applied Cryptography and Network Security*. Springer, 2022. doi:10.1007/978-3-031-09234-3_26.
- CGGI16. Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. Faster Fully Homomorphic Encryption: Bootstrapping in Less Than 0.1 Seconds. In *Advances in Cryptology – ASIACRYPT 2016*. Springer, 2016. doi:10.1007/978-3-662-53887-6_1.

- CHK⁺18. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. Bootstrapping for Approximate Homomorphic Encryption. In *Advances in Cryptology – EUROCRYPT 2018*. Springer, 2018. doi:10.1007/978-3-319-78381-9_14.
- CHK⁺19. Jung Hee Cheon, Kyoohyung Han, Andrey Kim, Miran Kim, and Yongsoo Song. A Full RNS Variant of Approximate Homomorphic Encryption. In *Selected Areas in Cryptography – SAC 2018*, pages 347–368. Springer, 2019. doi:10.1007/978-3-030-10970-7_16.
- CHKS25. Jung Hee Cheon, Guillaume Hanrot, Jongmin Kim, and Damien Stehlé. SHIP: A Shallow and Highly Parallelizable CKKS Bootstrapping Algorithm. In *Advances in Cryptology – EUROCRYPT 2025*. Springer, 2025. doi:10.1007/978-3-031-91131-6_14.
- CK25. Jean-Sébastien Coron and Robin Köstler. Low-Latency Bootstrapping for CKKS using Roots of Unity, 2025. <https://eprint.iacr.org/2025/651>.
- CKKS17. Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic Encryption for Arithmetic of Approximate Numbers. In *Advances in Cryptology – ASIACRYPT 2017*. Springer, 2017. doi:10.1007/978-3-319-70694-8_15.
- CLPX18. Hao Chen, Kim Laine, Rachel Player, and Yuhou Xia. High-precision arithmetic in homomorphic encryption. In *Topics in Cryptology – CT-RSA 2018*. Springer, 2018. doi:10.1007/978-3-319-76953-0_7.
- CS26. Jean-Sébastien Coron and Tim Seuré. PaCo: Bootstrapping for CKKS via Partial CoeffToSlot. In *Advances in Cryptology – ASIACRYPT 2025*. Springer, 2026. doi:10.1007/978-981-95-5122-4_2.
- DM15. Léo Ducas and Daniele Micciancio. FHEW: Bootstrapping Homomorphic Encryption in Less Than a Second. In *Advances in Cryptology – EUROCRYPT 2015*. Springer, 2015. doi:10.1007/978-3-662-46800-5_24.
- DMPS24. Nir Drucker, Guy Moshkovich, Tomer Pelleg, and Hayim Shaul. BLEACH: Cleaning Errors in Discrete Computations over CKKS. *Journal of Cryptology*, 37, 2024. doi:10.1007/s00145-023-09483-1.
- FV12. Junfeng Fan and Frederik Vercauteren. Somewhat Practical Fully Homomorphic Encryption, 2012. <https://eprint.iacr.org/2012/144>.
- Gen09. Craig Gentry. Fully Homomorphic Encryption Using Ideal Lattices. In *Proceedings of the Annual ACM Symposium on Theory of Computing*. ACM, 2009. doi:10.1145/1536414.1536440.
- GV25. Robin Geelen and Frederik Vercauteren. Fully Homomorphic Encryption for Cyclotomic Prime Moduli. In *Advances in Cryptology – EUROCRYPT 2025*. Springer, 2025. doi:10.1007/978-3-031-91131-6_13.
- GZ26. Mingyu Gao and Hongren Zheng. FHE for SIMD Arithmetic Logic Units with Amortized $O(1)$ Bootstrapping per Ciphertext. In *Advances in Cryptology – CRYPTO 2026*. Springer, 2026. doi:10.1007/978-3-032-35374-0_10.
- HHC19. Kyoohyung Han, Minki Hhan, and Jung Hee Cheon. Improved Homomorphic Discrete Fourier Transforms and FHE Bootstrapping. *IEEE Access*, 7:57361–57370, 2019. doi:10.1109/ACCESS.2019.2913850.
- HS14. Shai Halevi and Victor Shoup. Algorithms in HELib. In *Advances in Cryptology – CRYPTO 2014*. Springer, 2014. doi:10.1007/978-3-662-44371-2_31.
- HS18. Shai Halevi and Victor Shoup. Faster Homomorphic Linear Transformations in HELib. In *Advances in Cryptology – CRYPTO 2018*. Springer, 2018. doi:10.1007/978-3-319-96884-1_4.
- JL26. Daehyun Jang and Junho Lee. Arithmetic for Large-Characteristic Finite Fields in CKKS. Cryptology ePrint Archive, Paper 2026/1958, 2026. <https://eprint.iacr.org/2026/1958>.
- JPK⁺24. Jae Hyung Ju, Jaiyoung Park, Jongmin Kim, Minsik Kang, Donghwan Kim, Jung Hee Cheon, and Jung Ho Ahn. NeuJeans: Private Neural Network Inference with Joint Optimization of Convolution and FHE Bootstrapping. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. ACM, 2024. doi:10.1145/3658644.3690375.
- Kim25a. Jaehyung Kim. Bootstrapping GBFV with CKKS, 2025. <https://eprint.iacr.org/2025/888>.
- Kim25b. Jaehyung Kim. Efficient Homomorphic Integer Computer from CKKS. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2025(4):873–898, 2025. doi:10.46586/tches.v2025.i4.873-898.
- Kim26. Jaehyung Kim. Faster Homomorphic Integer Computer. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2026(3):1128–1151, 2026. doi:10.46586/tches.v2026.i3.1128-1151.
- KLN⁺26. Taeseong Kim, Jonghoo Lee, Taeyeong Noh, Jung Hee Cheon, and Guillaume Hanrot. Latency-Aware, High-Throughput Homomorphic AES Evaluation with CKKS. Cryptology ePrint Archive, Paper 2026/1209, 2026. <https://eprint.iacr.org/2026/1209>.
- KN25. Jaehyung Kim and Taeyeong Noh. Modular Reduction in CKKS. *IACR Communications in Cryptology*, 2(2), 2025. doi:10.62056/aeuvr-iuc.

- KPK⁺22. Seonghak Kim, Minji Park, Jaehyung Kim, Taekyung Kim, and Chohong Min. EvalRound Algorithm in CKKS Bootstrapping. In *Advances in Cryptology – ASIACRYPT 2022*. Springer, 2022. [doi:10.1007/978-3-031-22966-4_6](https://doi.org/10.1007/978-3-031-22966-4_6).
- Lat24. Lattigo v6. Online: <https://github.com/tuneinsight/lattigo>, August 2024. EPFL-LDS, Tune Insight SA.
- LM21. Baiyu Li and Daniele Micciancio. On the Security of Homomorphic Encryption on Approximate Numbers. In *Advances in Cryptology – EUROCRYPT 2021*. Springer, 2021. [doi:10.1007/978-3-030-77870-5_23](https://doi.org/10.1007/978-3-030-77870-5_23).
- LMSWS22. Baiyu Li, Daniele Micciancio, Mark Schultz-Wu, and Jessica Sorrell. Securing Approximate Homomorphic Encryption Using Differential Privacy. In *Advances in Cryptology – CRYPTO 2022*. Springer, 2022. [doi:10.1007/978-3-031-15802-5_20](https://doi.org/10.1007/978-3-031-15802-5_20).
- LMWW26. Jiaming Liu, Shihe Ma, Anyu Wang, and Xiaoyun Wang. High-Precision Homomorphic ALU over Arbitrary Moduli with $O(1)$ Bootstrapping, 2026. <https://eprint.iacr.org/2026/1869>.
- LPR10. Vadim Lyubashevsky, Chris Peikert, and Oded Regev. On Ideal Lattices and Learning with Errors over Rings. In *Advances in Cryptology – EUROCRYPT 2010*. Springer, 2010. [doi:10.1007/978-3-642-13190-5_1](https://doi.org/10.1007/978-3-642-13190-5_1).
- PZZ26. Chris Peikert, Doron Zarchy, and Guy Zyskind. High-Precision Exact FHE Made Simple, General, and Fast. In *Advances in Cryptology – CRYPTO 2026*. Springer, 2026. [doi:10.1007/978-3-032-35374-0_17](https://doi.org/10.1007/978-3-032-35374-0_17).
- Seu25. Tim Seuré. SCORE: A SlotToCoeff Optimization for Real-Vector Encryption in CKKS, 2025. <https://eprint.iacr.org/2025/1983>.
- SS26. Tim Seuré and Elias Suvanto. Finite-Field Arithmetic in CKKS. Cryptology ePrint Archive, Paper 2026/1102, 2026. <https://eprint.iacr.org/2026/1102>.
- SSTX09. Damien Stehlé, Ron Steinfeld, Keisuke Tanaka, and Keita Xagawa. Efficient Public Key Encryption Based on Ideal Lattices. In *Advances in Cryptology – ASIACRYPT 2009*. Springer, 2009. [doi:10.1007/978-3-642-10366-7_36](https://doi.org/10.1007/978-3-642-10366-7_36).
- WMGZ26. Yuchen Wei, Kaisheng Ma, Mingyu Gao, and Hongren Zheng. FHE for ALU over Large Prime Moduli and Application to One-Round Threshold ECDSA, 2026. <https://eprint.iacr.org/2026/1836>.

A Proofs for Lifting and Projection

We prove the results of [Section 4](#), using the notation and assumptions from that section.

Lemma 1 (Well-definedness of the flattening map). *The map $\text{Flatten}_n : \mathcal{R}_n \rightarrow \mathcal{R}_n$ is well-defined.*

Proof. By the coprimality assumption, there exist $u, v \in \mathbb{Q}[X]$ such that $ut + v(X^n + 1) = 1$. Thus, the image of u in $\mathcal{S}_n$ is the inverse of t , and division by t makes sense in $\mathcal{S}_n$. Moreover, the output of Flatten is in $\mathcal{R}_n$. Indeed, writing $D = P/t$ for $P \in \mathcal{R}_n$, there exists $I \in \mathcal{R}_n$ such that $[D]_1 = D + I$, and thus $\text{Flatten}_n(P) = t[D]_1 = t(D + I) = P + tI \in \mathcal{R}_n$. $\square$

Proposition 1 (Basic properties of the flattening map). *The map $\text{Flatten}_n : \mathcal{R}_n \rightarrow \mathcal{R}_n$ satisfies the following properties:*

- (a) *For every $P \in \mathcal{R}_n$, we have $(\text{Proj}_n \circ \text{Flatten}_n)(P) = \text{Proj}_n(P)$.*
- (b) *If $P, Q \in \mathcal{R}_n$ are such that $\text{Proj}_n(P) = \text{Proj}_n(Q)$, then $\text{Flatten}_n(P) = \text{Flatten}_n(Q)$.*
- (c) $\text{Flatten}_n \circ \text{Flatten}_n = \text{Flatten}_n$.

Proof. (a) There exists $I \in \mathcal{R}_n$ with $\text{Flatten}_n(P) = P + tI$, so $(\text{Proj}_n \circ \text{Flatten}_n)(P) = \text{Proj}_n(P) + \text{Proj}_n(tI) = \text{Proj}_n(P)$.

(b) If $\text{Proj}_n(P) = \text{Proj}_n(Q)$, then $P = Q + tI$ for some $I \in \mathcal{R}_n$, and thus $\text{Flatten}_n(P) = t[Q/t + I]_1 = t[Q/t]_1 = \text{Flatten}_n(Q)$.

(c) For $Q = \text{Flatten}_n(P)$, we have $\text{Proj}_n(Q) = \text{Proj}_n(P)$ by (a), and thus $\text{Flatten}_n(Q) = \text{Flatten}_n(P)$ by (b). $\square$

Proposition 2 (Projecting after lifting). *The map $\text{Proj}_n : \mathcal{R}_n \rightarrow \mathcal{A}_n$ is a left inverse of the encoding map $\text{Lift}_n : \mathcal{A}_n \rightarrow \mathcal{R}_n$:*

$$\text{Proj}_n \circ \text{Lift}_n = \text{id}_{\mathcal{A}_n}.$$

Proof. For $a \in \mathcal{A}_n$, we have $\text{Lift}_n(a) = \text{Flatten}_n(P)$ for any $P \in \mathcal{R}_n$ with $\text{Proj}_n(P) = a$. Then $(\text{Proj}_n \circ \text{Lift}_n)(a) = (\text{Proj}_n \circ \text{Flatten}_n)(P) = \text{Proj}_n(P) = a$, where the second equality is [Proposition 1 \(a\)](#). $\square$

Proposition 3 (Lifting bounds). *For every $a \in \mathcal{A}_n$, it holds that:*

$$\left\| \frac{\text{Lift}_n(a)}{t} \right\|_{\infty} \leq \frac{1}{2}, \quad \|\text{Lift}_n(a)\|_{\infty} \leq \frac{\|t\|_1}{2}.$$

Proof. For any $P \in \mathcal{R}_n$ with $\text{Proj}_n(P) = a$, the definition of Lift_n gives $\text{Lift}_n(a)/t = [P/t]_1$, i.e., $\text{Lift}_n(a)/t$ has coefficients in the interval $(-1/2, 1/2]$. For the second inequality, note that multiplication by each monomial X^j preserves the coefficient infinity norm in $\mathcal{S}_n$, so the triangle inequality gives $\|tP\|_{\infty} \leq \|t\|_1 \|P\|_{\infty}$ for all $P \in \mathcal{S}_n$, and thus $\|\text{Lift}_n(a)\|_{\infty} = \|t \text{Lift}_n(a)/t\|_{\infty} \leq \|t\|_1 \|\text{Lift}_n(a)/t\|_{\infty} \leq \|t\|_1/2$. $\square$

Proposition 4 (Lifting respects arithmetic operations). *Let $a, a' \in \mathcal{A}_n$ and let $P = \text{Lift}_n(a) + tI$ and $P' = \text{Lift}_n(a') + tI'$ for some $I, I' \in \mathcal{R}_n$. Then $P + P' = \text{Lift}_n(a + a') + tI_{\text{add}}$ and $PP' = \text{Lift}_n(aa') + tI_{\text{mul}}$ for some $I_{\text{add}}, I_{\text{mul}} \in \mathcal{R}_n$.*

Proof. We only show the multiplicative case; the additive case is similar. We have $\text{Proj}_n(PP' - \text{Lift}_n(aa')) = \text{Proj}_n(P)\text{Proj}_n(P') - \text{Proj}_n(\text{Lift}_n(aa')) = aa' - aa' = 0$ by [Proposition 2](#), so $PP' - \text{Lift}_n(aa') \in \ker(\text{Proj}_n) = \langle t \rangle$. Hence, there exists $I_{\text{mul}} \in \mathcal{R}_n$ such that $PP' - \text{Lift}_n(aa') = tI_{\text{mul}}$. $\square$

B Proof for Residue-Ring Decomposition

We use the notation of [Section 5](#): n is a power of two, $\mathcal{R}_n = \mathbb{Z}[X]/\langle X^n + 1 \rangle$, and $\mathcal{A}_n = \mathcal{R}_n/\langle t \rangle$.

Proposition 5 (Residue-ring decomposition). *Let $t = X^r - \beta^r$, where $r \in \llbracket 1, n/2 \rrbracket$ is a power of two and $\beta \geq 2$ is an integer. If $r = 1$ or if β is even, we have an isomorphism of rings $\mathcal{A}_n \cong \mathbb{Z}_M^r$ for $M = \beta^n + 1$.*

Proof. We first have $\mathcal{A}_n \cong \mathbb{Z}[X]/\langle X^n + 1, X^r - \beta^r \rangle$. Since r divides n , it holds that $X^n \equiv_t (X^r)^{n/r} \equiv_t \beta^n$ and hence that $X^n + 1 \equiv_t M$. It follows that $\mathcal{A}_n \cong \mathbb{Z}_M[X]/\langle X^r - \beta^r \rangle$, which is isomorphic to $\mathbb{Z}_M$ if $r = 1$.

Assume now that $r > 1$ and that β is even. Let $\omega = \beta^{2n/r}$; its multiplicative order modulo M is r . Moreover, for every $k \in \llbracket 1, r-1 \rrbracket$, the element $\omega^k - 1$ is invertible modulo M . Indeed, suppose that a prime p divides both M and $\omega^k - 1$. Since β is even, it follows that M is odd, and thus $p \neq 2$. We have $\beta^n \equiv_p -1 \not\equiv_p 1$ since p divides M , so the multiplicative order of β modulo p is $2n$ (as n is a power of two). On the other hand, we have $\beta^{2nk/r} = \omega^k \equiv_p 1$ since p divides $\omega^k - 1$, so the multiplicative order of β modulo p divides $2nk/r$. This means that $2n$ divides $2nk/r$, i.e., that r divides $k \in \llbracket 1, r-1 \rrbracket$, a contradiction.

Since ω and β are invertible modulo M , the elements $\beta, \beta\omega, \dots, \beta\omega^{r-1}$ have pairwise differences that are invertible modulo M by the above argument. Therefore, we have $X^r - \beta^r \equiv_M \prod_{j=0}^{r-1} (X - \beta\omega^j)$ (both polynomials are monic of degree r and have the same r roots $\beta\omega^j$ modulo M , which in turn

have pairwise differences that are invertible modulo M). It follows that the ideals $\langle X - \beta\omega^j \rangle$ are pairwise coprime in $\mathbb{Z}_M[X]$, and the *Chinese remainder theorem* thus gives:

$$\mathcal{A}_n \cong \frac{\mathbb{Z}_M[X]}{\langle X^r - \beta^r \rangle} \cong \prod_{j=0}^{r-1} \frac{\mathbb{Z}_M[X]}{\langle X - \beta\omega^j \rangle} \cong \mathbb{Z}_M^r. \quad \square$$

C Proofs for Our Framework within CKKS

We first establish two bounds on the packing maps, then restate and prove the results of [Section 6](#). We use the notation and assumptions of that section. In particular, $t \in \mathbb{Z}[X]$ has degree less than n and is coprime to $X^n + 1$ in $\mathbb{Q}[X]$, and $\|t\|_1$ denotes the coefficient 1-norm of its image in $\mathcal{R}_n$. We use the bounds $\|tP\|_\infty \leq \|t\|_1 \|P\|_\infty$ and $\|PQ\|_\infty \leq n \|P\|_\infty \|Q\|_\infty$ for $P, Q \in \mathcal{S}_n$, also applied componentwise to polynomial lanes.

Lemma 2 (Packing isomorphisms are scaled isometries). *For all divisors $d, d' \geq 2$ of N , the map $\sqrt{d'/d} \cdot \Phi_{d' \leftarrow d} : \mathcal{S}_d^{N/d} \rightarrow \mathcal{S}_{d'}^{N/d'}$ is an isometry with respect to the 2-norms on $\mathcal{S}_d^{N/d}$ and $\mathcal{S}_{d'}^{N/d'}$. In other words, we have $\|\Phi_{d' \leftarrow d}(\mathbf{P})\|_2 = \sqrt{d'/d} \cdot \|\mathbf{P}\|_2$ for every $\mathbf{P} \in \mathcal{S}_d^{N/d}$.*

Proof. The map $\sqrt{2/d} \cdot \text{eval}_d : \mathcal{S}_d \rightarrow \mathbb{C}^{d/2}$ is an isometry with respect to the 2-norms on $\mathcal{S}_d$ and $\mathbb{C}^{d/2}$ by orthogonality of the roots of unity; see [\[CKKS17, Sections 2.2 and 3.2\]](#) for the canonical embedding. Consequently, the same holds true for the extended map $\sqrt{2/d} \cdot \text{eval}_d : \mathcal{S}_d^{N/d} \rightarrow \mathbb{C}^{N/2}$ and the inverse map $\sqrt{d'/2} \cdot \text{eval}_{d'}^{-1} : \mathbb{C}^{N/2} \rightarrow \mathcal{S}_{d'}^{N/d'}$, and hence for their composition $\sqrt{d'/d} \cdot \Phi_{d' \leftarrow d} : \mathcal{S}_d^{N/d} \rightarrow \mathcal{S}_{d'}^{N/d'}$. $\square$

Lemma 3 (Infinity norms under packing isomorphisms). *For all divisors $d, d' \geq 2$ of N and every $\mathbf{P} \in \mathcal{S}_d^{N/d}$, we have:*

$$\|\Phi_{d' \leftarrow d}(\mathbf{P})\|_\infty \leq \sqrt{\frac{Nd}{d'}} \|\mathbf{P}\|_\infty.$$

Proof. By [Lemma 2](#), we have:

$$\|\Phi_{d' \leftarrow d}(\mathbf{P})\|_\infty \leq \|\Phi_{d' \leftarrow d}(\mathbf{P})\|_2 = \sqrt{\frac{d}{d'}} \|\mathbf{P}\|_2 \leq \sqrt{\frac{d}{d'}} \sqrt{N} \|\mathbf{P}\|_\infty. \quad \square$$

Proposition 8 (Correct decryption). *Let $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Then $(\text{Dcd}_{\mathcal{A}_n} \circ \text{Dec}_{s,\ell})(\mathbf{ct}) = \mathbf{a}$ if the following two inequalities are satisfied:*

$$\sqrt{n} \cdot \left(\Delta \|t\|_1 \left(\frac{1}{2} + B_I \right) + B_e \right) < \frac{q_\ell}{2}, \quad B_e < \frac{\Delta}{2}.$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ witness $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Using that $\|\text{Lift}_n(\mathbf{a})\|_\infty \leq \|t\|_1/2$ by [Proposition 3](#), that $\|t\mathbf{I}\|_\infty \leq \|t\|_1 B_I$ and that $\|\mathbf{e}\|_\infty \leq B_e$, [Lemma 3](#) gives:

$$\left\| \Phi_{N \leftarrow n}(\Delta \text{Lift}_n(\mathbf{a}) + \Delta t\mathbf{I} + \mathbf{e}) \right\|_\infty \leq \sqrt{n} \cdot \left(\Delta \|t\|_1 \left(\frac{1}{2} + B_I \right) + B_e \right) < \frac{q_\ell}{2}.$$

It follows that $\text{Dec}_{s,\ell}(\mathbf{ct}) = \Phi_{N \leftarrow n}(\Delta \text{Lift}_n(\mathbf{a}) + \Delta t\mathbf{I} + \mathbf{e})$ (that is, we have equality, and not only equivalence modulo q_ℓ). Now, $(\text{Dcd}_{\mathcal{A}_n} \circ \text{Dec}_{s,\ell})(\mathbf{ct}) = \mathbf{a}$ follows because $\|\mathbf{e}\|_\infty \leq B_e < \Delta/2$. $\square$

Proposition 9 (Fresh encryption). Let $\mathbf{a} \in \mathcal{A}_n^{N/n}$ and $P = \Delta\Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}))$. Then a ciphertext $\mathbf{ct} \leftarrow \text{Enc}_s(\lfloor P \rfloor)$ satisfies $\mathbf{ct} \in \mathcal{C}_{s,L}(\mathbf{a}; B_I, B_e)$, where:

$$B_I = 0, \quad B_e = B_{\text{enc},n} + \frac{N}{2\sqrt{n}}.$$

Proof. Since $\lfloor P \rfloor = P - \lfloor P \rfloor_1$, the encryption relation gives:

$$\text{Dec}_{s,L}(\mathbf{ct}) \equiv_{q_L} P - \lfloor P \rfloor_1 + e_{\text{enc}} = \Phi_{N \leftarrow n}(\Delta\text{Lift}_n(\mathbf{a}) + \mathbf{e}),$$

where $\mathbf{e} = \Phi_{n \leftarrow N}(e_{\text{enc}} - \lfloor P \rfloor_1)$. Thus, $(\mathbf{0}, \mathbf{e})$ witnesses the claimed membership. By the triangle inequality, Lemma 3, and the coefficientwise rounding bound $\|\lfloor P \rfloor_1\|_\infty \leq 1/2$:

$$\|\mathbf{e}\|_\infty \leq \|\Phi_{n \leftarrow N}(e_{\text{enc}})\|_\infty + \|\Phi_{n \leftarrow N}(\lfloor P \rfloor_1)\|_\infty \leq B_{\text{enc},n} + \frac{N}{2\sqrt{n}} = B_e. \quad \square$$

Proposition 10 (Plaintext–ciphertext addition). Consider a ciphertext $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and a plaintext $\mathbf{a}' \in \mathcal{A}_n^{N/n}$. Set $P = \Delta\Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}'))$ and $B'_e = \|\Phi_{n \leftarrow N}(\lfloor P \rfloor_1)\|_\infty$. Then $\mathbf{ct}_{\text{add}} \in \mathcal{C}_{s,\ell}(\mathbf{a} + \mathbf{a}'; B_{I,\text{add}}, B_{e,\text{add}})$, where $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}(\lfloor P \rfloor, \mathbf{ct})$ and:

$$B_{I,\text{add}} = B_I + 1, \quad B_{e,\text{add}} = B_e + B'_e.$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ witness $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. By Proposition 4, $\text{Lift}_n(\mathbf{a}) + \text{Lift}_n(\mathbf{a}') = \text{Lift}_n(\mathbf{a} + \mathbf{a}') + t\mathbf{J}$ for some $\mathbf{J} \in \mathcal{R}_n^{N/n}$. Moreover, by isolating $\mathbf{J}$, Proposition 3 gives $\|\mathbf{J}\|_\infty \leq 3/2$, and hence $\|\mathbf{J}\|_\infty \leq 1$ since $\mathbf{J}$ has integer coefficients. Setting $\mathbf{I}_{\text{add}} = \mathbf{I} + \mathbf{J}$ and $\mathbf{e}_{\text{add}} = \mathbf{e} - \Phi_{n \leftarrow N}(\lfloor P \rfloor_1)$, and using $\lfloor P \rfloor = P - \lfloor P \rfloor_1$, we obtain:

$$\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{add}}) \equiv_{q_\ell} \Phi_{N \leftarrow n}(\Delta\text{Lift}_n(\mathbf{a} + \mathbf{a}') + \Delta t\mathbf{I}_{\text{add}} + \mathbf{e}_{\text{add}}).$$

Finally, $\|\mathbf{I}_{\text{add}}\|_\infty \leq B_I + 1$ and $\|\mathbf{e}_{\text{add}}\|_\infty \leq B_e + B'_e$, proving the claim. $\square$

Proposition 11 (Ciphertext–ciphertext addition). Consider ciphertexts $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$. Then $\mathbf{ct}_{\text{add}} \in \mathcal{C}_{s,\ell}(\mathbf{a} + \mathbf{a}'; B_{I,\text{add}}, B_{e,\text{add}})$, where $\mathbf{ct}_{\text{add}} \leftarrow \text{Add}(\mathbf{ct}, \mathbf{ct}')$ and:

$$B_{I,\text{add}} = B_I + B'_I + 1, \quad B_{e,\text{add}} = B_e + B'_e.$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ and $(\mathbf{I}', \mathbf{e}')$ witness that $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$, respectively. We can again write $\text{Lift}_n(\mathbf{a}) + \text{Lift}_n(\mathbf{a}') = \text{Lift}_n(\mathbf{a} + \mathbf{a}') + t\mathbf{J}$ for some $\mathbf{J} \in \mathcal{R}_n^{N/n}$ with $\|\mathbf{J}\|_\infty \leq 1$. Setting $\mathbf{I}_{\text{add}} = \mathbf{I} + \mathbf{I}' + \mathbf{J}$ and $\mathbf{e}_{\text{add}} = \mathbf{e} + \mathbf{e}'$, we obtain:

$$\text{Dec}_{s,\ell}(\mathbf{ct}_{\text{add}}) \equiv_{q_\ell} \Phi_{N \leftarrow n}(\Delta\text{Lift}_n(\mathbf{a} + \mathbf{a}') + \Delta t(\mathbf{I} + \mathbf{I}' + \mathbf{J}) + \mathbf{e} + \mathbf{e}').$$

Finally, $\|\mathbf{I}_{\text{add}}\|_\infty \leq B_I + B'_I + 1$ and $\|\mathbf{e}_{\text{add}}\|_\infty \leq B_e + B'_e$, proving the claim. $\square$

Proposition 12 (Plaintext–ciphertext multiplication). For a level $\ell \in \llbracket 1, L \rrbracket$, consider a ciphertext $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and a plaintext $\mathbf{a}' \in \mathcal{A}_n^{N/n}$. Set $P = \Delta\Phi_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}'))$ and $B'_e = \|\Phi_{n \leftarrow N}(\lfloor P \rfloor_1)\|_\infty$. Then $\mathbf{ct}_{\text{mul}} \in \mathcal{C}_{s,\ell-1}(\mathbf{a} \cdot \mathbf{a}'; B_{I,\text{mul}}, B_{e,\text{mul}})$, where $\mathbf{ct}_{\text{mul}} \leftarrow \text{Rescale}(\text{Mul}(\lfloor P \rfloor, \mathbf{ct}))$ and:

$$B_{I,\text{mul}} = \frac{n\|t\|_1}{2} \left(\frac{1}{2} + B_I \right) + \frac{1}{2},$$

$$B_{e,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + \frac{n\|t\|_1}{2} B_e + \frac{nB_e B'_e}{\Delta} + B_{\text{rs},n}.$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ witness $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$, and set $\mathbf{Q} = \text{Lift}_n(\mathbf{a}) + t\mathbf{I}$, $\mathbf{Q}' = \text{Lift}_n(\mathbf{a}')$ and $\mathbf{e}' = -\Phi_{n \leftarrow N}([P]_1)$. By [Proposition 4](#), we have $\mathbf{Q}\mathbf{Q}' = \text{Lift}_n(\mathbf{a} \cdot \mathbf{a}') + t\mathbf{I}_{\text{mul}}$ for some $\mathbf{I}_{\text{mul}} \in \mathcal{R}_n^{N/n}$. Isolating $\mathbf{I}_{\text{mul}}$ gives:

$$\mathbf{I}_{\text{mul}} = \left(\frac{\text{Lift}_n(\mathbf{a})}{t} + \mathbf{I} \right) \mathbf{Q}' - \frac{\text{Lift}_n(\mathbf{a} \cdot \mathbf{a}')}{t}.$$

Using [Proposition 3](#) and that $\|fg\|_\infty \leq n\|f\|_\infty\|g\|_\infty$ for all $f, g \in \mathcal{S}_n$, we obtain $\|\mathbf{I}_{\text{mul}}\|_\infty \leq B_{I,\text{mul}}$.

Since $[P] = \Phi_{N \leftarrow n}(\Delta\mathbf{Q}' + \mathbf{e}')$ and $\Phi_{N \leftarrow n}$ is a ring isomorphism, multiplication followed by rescaling gives:

$$\text{Dec}_{s,\ell-1}(\mathbf{ct}_{\text{mul}}) \equiv_{q_{\ell-1}} \Phi_{N \leftarrow n}(\Delta\text{Lift}_n(\mathbf{a} \cdot \mathbf{a}') + \Delta t\mathbf{I}_{\text{mul}} + \mathbf{e}_{\text{mul}}),$$

where $\mathbf{e}_{\text{mul}} = \mathbf{Q}\mathbf{e}' + \mathbf{Q}'\mathbf{e} + \mathbf{e}\mathbf{e}'/\Delta + \Phi_{n \leftarrow N}(e_{rs})$. Finally, using $\|\mathbf{Q}\|_\infty \leq \|t\|_1(1/2 + B_I)$, $\|\mathbf{Q}'\|_\infty \leq \|t\|_1/2$ and $\|\mathbf{e}'\|_\infty = B'_e$, we obtain:

$$\|\mathbf{e}_{\text{mul}}\|_\infty \leq n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + \frac{n\|t\|_1}{2} B_e + \frac{nB_e B'_e}{\Delta} + B_{rs,n} = B_{e,\text{mul}}. \quad \square$$

Proposition 13 (Ciphertext–ciphertext multiplication). *For a level $\ell \in \llbracket 1, L \rrbracket$, consider ciphertexts $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$. Then $\mathbf{ct}_{\text{mul}} \in \mathcal{C}_{s,\ell-1}(\mathbf{a} \cdot \mathbf{a}'; B_{I,\text{mul}}, B_{e,\text{mul}})$, where $\mathbf{ct}_{\text{mul}} \leftarrow \text{Rescale}(\text{Mul}(\mathbf{ct}, \mathbf{ct}'))$ and:*

$$B_{I,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) \left(\frac{1}{2} + B'_I \right) + \frac{1}{2},$$

$$B_{e,\text{mul}} = n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + n\|t\|_1 \left(\frac{1}{2} + B'_I \right) B_e + \frac{nB_e B'_e + B_{ks,n}}{\Delta} + B_{rs,n}.$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ and $(\mathbf{I}', \mathbf{e}')$ witness $\mathbf{ct} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$ and $\mathbf{ct}' \in \mathcal{C}_{s,\ell}(\mathbf{a}'; B'_I, B'_e)$, respectively. Set $\mathbf{Q} = \text{Lift}_n(\mathbf{a}) + t\mathbf{I}$ and $\mathbf{Q}' = \text{Lift}_n(\mathbf{a}') + t\mathbf{I}'$. By [Proposition 4](#), we have $\mathbf{Q}\mathbf{Q}' = \text{Lift}_n(\mathbf{a} \cdot \mathbf{a}') + t\mathbf{I}_{\text{mul}}$ for some $\mathbf{I}_{\text{mul}} \in \mathcal{R}_n^{N/n}$. Isolating $\mathbf{I}_{\text{mul}}$ gives:

$$\mathbf{I}_{\text{mul}} = \left(\frac{\text{Lift}_n(\mathbf{a})}{t} + \mathbf{I} \right) \mathbf{Q}' - \frac{\text{Lift}_n(\mathbf{a} \cdot \mathbf{a}')}{t}.$$

Using [Proposition 3](#) and the coefficient bound $\|fg\|_\infty \leq n\|f\|_\infty\|g\|_\infty$ for all $f, g \in \mathcal{S}_n$, we obtain $\|\mathbf{I}_{\text{mul}}\|_\infty \leq B_{I,\text{mul}}$. Since $\Phi_{N \leftarrow n}$ is a ring isomorphism, multiplication with relinearization followed by rescaling gives:

$$\text{Dec}_{s,\ell-1}(\mathbf{ct}_{\text{mul}}) \equiv_{q_{\ell-1}} \Phi_{N \leftarrow n}(\Delta\text{Lift}_n(\mathbf{a} \cdot \mathbf{a}') + \Delta t\mathbf{I}_{\text{mul}} + \mathbf{e}_{\text{mul}}),$$

where $\mathbf{e}_{\text{mul}} = \mathbf{Q}\mathbf{e}' + \mathbf{Q}'\mathbf{e} + (\mathbf{e}\mathbf{e}' + \Phi_{n \leftarrow N}(e_{ks}))/\Delta + \Phi_{n \leftarrow N}(e_{rs})$. Finally, using $\|\mathbf{Q}\|_\infty \leq \|t\|_1(1/2 + B_I)$ and $\|\mathbf{Q}'\|_\infty \leq \|t\|_1(1/2 + B'_I)$, we obtain:

$$\|\mathbf{e}_{\text{mul}}\|_\infty \leq n\|t\|_1 \left(\frac{1}{2} + B_I \right) B'_e + n\|t\|_1 \left(\frac{1}{2} + B'_I \right) B_e + \frac{nB_e B'_e + B_{ks,n}}{\Delta} + B_{rs,n} = B_{e,\text{mul}}. \quad \square$$

D Correctness Proof for Lazy Flattening

We use the notation and transformation-error bounds of the lazy-flattening subsection. In particular, $t \in \mathbb{Z}[X]$ has degree less than n and is coprime to $X^n + 1$ in $\mathbb{Q}[X]$.

Proposition 14 (Correctness of lazy flattening). Set $\ell = \ell_{N \leftarrow n}$ and $\tilde{\ell} = L - \ell_{n \leftarrow N}$, and suppose $\tilde{\ell} \geq 0$. Consider $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Then [Algorithm 1](#) returns $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s,\tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$, where:

$$\begin{aligned}\tilde{B}_I &= \frac{h'}{2} + 1 + \frac{\|\frac{1}{t}\|_1 B_e}{\Delta} + \frac{B_{N \leftarrow n} + B_{\text{ks}}}{q_0}, \\ \tilde{B}_e &= B_e + \frac{\Delta \|t\|_1}{q_0} (B_{N \leftarrow n} + 2B_{\text{ks}}) + B_{n \leftarrow N}.\end{aligned}$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ witness the input membership. After [line 2](#), the ciphertext decrypts (modulo q_0) to $q_0 \kappa_{N \leftarrow n}(\text{Lift}_n(\mathbf{a})/t) + e_{\text{pre}}$, where e_{pre} collects the errors before modulus raising:

$$e_{\text{pre}} = \frac{q_0}{\Delta} \kappa_{N \leftarrow n} \left(\frac{\mathbf{e}}{t} \right) + e_{N \leftarrow n} + e_{\text{ks}};$$

the contribution $q_0 \kappa_{N \leftarrow n}(\mathbf{I})$ vanishes modulo q_0 . Writing this ciphertext as $\mathbf{ct} = (\mathbf{ct}_0, \mathbf{ct}_1)$ with centered coefficients modulo q_0 , let $m_{\text{mr}} = \mathbf{ct}_0 s' + \mathbf{ct}_1$ be its unreduced decryption polynomial. It satisfies $\|m_{\text{mr}}\|_\infty \leq q_0(h' + 1)/2$, and for some $\tilde{\mathbf{I}} \in \mathcal{R}_n^{N/n}$ we have:

$$m_{\text{mr}} = q_0 \kappa_{N \leftarrow n}(\text{Lift}_n(\mathbf{a})/t) + e_{\text{pre}} + q_0 \kappa_{N \leftarrow n}(\tilde{\mathbf{I}}).$$

The vector $\tilde{\mathbf{I}}$ will be the output overflow. By [Proposition 3](#), the error bounds, and coefficient-norm preservation under $\kappa_{N \leftarrow n}$:

$$\|\tilde{\mathbf{I}}\|_\infty \leq \frac{h' + 2}{2} + \frac{\|e_{\text{pre}}\|_\infty}{q_0} \leq \frac{h'}{2} + 1 + \frac{\|\frac{1}{t}\|_1 B_e}{\Delta} + \frac{B_{N \leftarrow n} + B_{\text{ks}}}{q_0} = \tilde{B}_I.$$

[Line 3](#) preserves m_{mr} modulo q_L , while [line 4](#) adds e'_{ks} . At output level $\tilde{\ell}$, the reverse transformation in [line 5](#) gives:

$$\text{Dec}_{s,\tilde{\ell}}(\mathbf{ct}_{\text{out}}) \equiv_{q_{\tilde{\ell}}} \Phi_{N \leftarrow n}(\Delta \text{Lift}_n(\mathbf{a}) + \Delta t \tilde{\mathbf{I}} + \tilde{\mathbf{e}}),$$

where:

$$\tilde{\mathbf{e}} = \mathbf{e} + \frac{\Delta}{q_0} t \kappa_{n \leftarrow N}(e_{N \leftarrow n} + e_{\text{ks}} + e'_{\text{ks}}) + \mathbf{e}_{n \leftarrow N}.$$

Finally, $\kappa_{n \leftarrow N}$ preserves coefficient norms and $\|tf\|_\infty \leq \|t\|_1 \|f\|_\infty$ for every $f \in \mathcal{S}_n$. Applying this inequality in each lane gives $\|\tilde{\mathbf{e}}\|_\infty \leq \tilde{B}_e$, as required. $\square$

E Correctness Proof for Arithmetic Bootstrapping

We use the notation and error bounds of the arithmetic bootstrapping subsection.

Proposition 15 (Correctness of arithmetic bootstrapping). Set $\ell = \ell_{N \leftarrow n}$ and $\tilde{\ell} = L - \ell_{2 \leftarrow N} - \ell_{\text{em}} - \ell_{n \leftarrow 2}$, and suppose $\tilde{\ell} \geq 0$ and $L \geq \tilde{\ell} + \ell_{n \leftarrow N}$. Consider $\mathbf{ct}_{\text{in}} \in \mathcal{C}_{s,\ell}(\mathbf{a}; B_I, B_e)$. Assume the following condition:

$$\frac{B_e + B_{2 \leftarrow N}}{\Delta} + \frac{\|t\|_1 (B_{N \leftarrow n} + B_{\text{ks}}) + B_{\text{ks}}}{q_0} \leq \varepsilon_{\text{em}}. \quad (1)$$

Then [Algorithm 2](#) returns $\mathbf{ct}_{\text{out}} \in \mathcal{C}_{s,\tilde{\ell}}(\mathbf{a}; \tilde{B}_I, \tilde{B}_e)$, where:

$$\begin{aligned}\tilde{B}_I &= \frac{h'}{2} + 1 + \frac{\|\frac{1}{t}\|_1 B_e}{\Delta} + \frac{B_{N \leftarrow n} + B_{\text{ks}}}{q_0}, \\ \tilde{B}_e &= B_{n \leftarrow N} + B_{2 \leftarrow N} + B_{\text{em}} + B_{n \leftarrow 2} + \frac{2\Delta}{q_0} B_{\text{ks}}.\end{aligned}$$

Proof. Let $(\mathbf{I}, \mathbf{e})$ witness the input membership. Up to [line 3](#), the algorithm follows lazy flattening with top level $\tilde{\ell} + \ell_{n \leftarrow N}$. By its proof, after the exact multiplication in [line 4](#), the ciphertext under s' decrypts modulo $q_{\tilde{\ell} + \ell_{n \leftarrow N}}$ to $q_0 P + E$, where $P = \kappa_{N \leftarrow n}(\text{Lift}_n(\mathbf{a}) + t\tilde{\mathbf{I}}) \in \mathcal{R}_N$, $\|\tilde{\mathbf{I}}\|_\infty \leq \tilde{B}_I$, and:

$$E = \frac{q_0}{\Delta} \kappa_{N \leftarrow n}(\mathbf{e}) + t(X^{N/n})(e_{N \leftarrow n} + e_{\text{ks}}^{\text{in}}).$$

Here $e_{\text{ks}}^{\text{in}}$ denotes the error of the initial key switch to s' , and we use $\kappa_{N \leftarrow n}(t\mathbf{F}) = t(X^{N/n})\kappa_{N \leftarrow n}(\mathbf{F})$ for $\mathbf{F} \in \mathcal{S}_n^{N/n}$.

[Line 5](#) saves the error branch under s' and removes $q_0 P$ modulo q_0 . In the error branch, the ciphertext is already under s' , so [line 6](#) raises its modulus directly. After the key switch in [line 7](#), it decrypts to $E + e_{\text{ks}} + q_0 K$ modulo q_L for some $K \in \mathcal{R}_N$, where e_{ks} is the key-switching error in the error branch and $\|e_{\text{ks}}\|_\infty \leq B_{\text{ks}}$. [Line 8](#) exposes the coefficients in $\mathcal{S}_2^{N/2}$ at scale Δ . Condition [\(1\)](#) ensures that each normalized coefficient differs from the corresponding integer coefficient of $\kappa_{2 \leftarrow N}(K)$ by at most $\varepsilon_{\text{em}} < 1/2$, including the transformation error. Moreover, the centered ciphertext at [line 6](#) has unreduced decryption $E + q_0 K$ of infinity norm at most $q_0(h' + 1)/2$. Since [\(1\)](#) also gives $\|E\|_\infty/q_0 \leq \varepsilon_{\text{em}} < 1/2$, we obtain:

$$\|K\|_\infty \leq \frac{h' + 1}{2} + \frac{\|E\|_\infty}{q_0} < \frac{h'}{2} + 1.$$

All real and imaginary components therefore lie in the prescribed domain, so [line 9](#) removes the integer contribution with additional error bounded by B_{em} . After [line 10](#), the error branch represents:

$$\frac{\Delta}{q_0} \kappa_{n \leftarrow N}(E + e_{\text{ks}}) + \mathbf{e}_{\text{ext}}, \quad \|\mathbf{e}_{\text{ext}}\|_\infty \leq B_{2 \leftarrow N} + B_{\text{em}} + B_{n \leftarrow 2}.$$

The errors add without norm amplification because $\kappa_{n \leftarrow 2}$ only permutes coefficients.

The retained branch is then switched to s in [line 11](#), introducing a key-switching error e'_{ks} with $\|e'_{\text{ks}}\|_\infty \leq B_{\text{ks}}$. Then $\mathbf{W}_{n \leftarrow N}$ in [line 12](#) maps the retained representative to $\Delta \kappa_{n \leftarrow N}(P) + (\Delta/q_0) \kappa_{n \leftarrow N}(E + e'_{\text{ks}}) + \mathbf{e}_{n \leftarrow N}$, with $\|\mathbf{e}_{n \leftarrow N}\|_\infty \leq B_{n \leftarrow N}$.

Subtraction in [line 13](#) cancels E , leaving the lane representative $\Delta \text{Lift}_n(\mathbf{a}) + \Delta t\tilde{\mathbf{I}} + \tilde{\mathbf{e}}$, where:

$$\tilde{\mathbf{e}} = \mathbf{e}_{n \leftarrow N} - \mathbf{e}_{\text{ext}} + \frac{\Delta}{q_0} \kappa_{n \leftarrow N}(e'_{\text{ks}} - e_{\text{ks}}).$$

Coefficient-norm preservation under $\kappa_{n \leftarrow N}$ gives $\|\tilde{\mathbf{e}}\|_\infty \leq \tilde{B}_e$. The level assumptions allow both branches to be reduced to $q_{\tilde{\ell}}$, proving the claimed output membership. $\square$

F Implementation Details and Additional Precision Measurements

This appendix describes the implementation used in [Section 8](#), using the notation of [Section 7](#). The implementation adjusts the first transformation to the actual input scale, allows a lower output level, and uses the key-switching schedule described below.

F.1 Secret Keys and Levels

The main secret s is uniform ternary. Input and output ciphertexts, rotation keys, and the re-linearization key are encrypted under s . The ephemeral secret s' is ternary of Hamming weight $h' = 32$ and is used during modulus raising. As in [Section 8](#), write P_{aux} for the auxiliary modulus and $p_{\text{aux},0}$ for one of its primes. The key-switching key from s to s' is generated modulo $q_0 p_{\text{aux},0}$, with $\log_2(q_0 p_{\text{aux},0}) = 121$ in our parameters. It is the only evaluation key encrypted under s' ; the key-switching keys from s' back to s are encrypted under s .

We retain the level costs $\ell_{N \leftarrow n}$, $\ell_{n \leftarrow N}$, $\ell_{2 \leftarrow N}$, $\ell_{n \leftarrow 2}$, and ℓ_{em} from [Section 7](#), and let L be the top level of the modulus chain. The input to the first transformation is at level $\ell = \ell_{N \leftarrow n}$. If an input arrives at a higher level, the implementation first reduces it modulo q_ℓ . We write $\tilde{\ell}$ for the chosen output level, which may be lower than the output level in the corresponding algorithm. Both procedures raise the retained ciphertext to $\tilde{\ell} + \ell_{n \leftarrow N}$, with $0 \leq \tilde{\ell}$ and $\tilde{\ell} + \ell_{n \leftarrow N} \leq L$. For Boot, define the level available from the error branch by:

$$\hat{\ell} = L - \ell_{2 \leftarrow N} - \ell_{\text{em}} - \ell_{n \leftarrow 2}.$$

We additionally require $\tilde{\ell} \leq \hat{\ell}$ and reduce the error branch modulo $q_{\tilde{\ell}}$ before subtraction. For example, at $n = 32$, the implementation chooses $\tilde{\ell} = 3$ while $\hat{\ell} = 4$.

As in [Section 3.5](#), a transformation evaluated as ρ factors consumes ρ levels. Grouping j consecutive Fourier stages produces a matrix with about 2^j diagonals. Thus, reducing the number of levels increases the cost of evaluating each factor.

F.2 Input Scale and Refreshing Procedures

Let S be the actual scale of the input ciphertext. For example, multiplying two ciphertexts at scale Δ and rescaling from level j to $j - 1$ gives $S = \Delta^2/p_j$, where $p_j = q_j/q_{j-1}$ is the removed prime. We therefore use the following scale-adjusted forward transformation:

$$\mathbf{V}_{N \leftarrow n}^{(S)} := \frac{q_0}{S} \cdot \mathbf{U}_{N \leftarrow n} \mathbf{D}_t^{-1} = \frac{\Delta}{S} \cdot \mathbf{V}_{N \leftarrow n}.$$

It agrees with $\mathbf{V}_{N \leftarrow n}$ when $S = \Delta$. Using the unadjusted matrix would leave a residual overflow term $q_0(S/\Delta - 1)\kappa_{N \leftarrow n}(\mathbf{I})$ after reduction modulo q_0 ; in our parameters, this causes a normalized error of about 2^{-39} after one multiplication. Both refreshing procedures return a ciphertext at scale Δ . The evaluation loop therefore encounters only a few distinct input scales, whose transformation encodings are cached.

[Table 7](#) gives the implemented lazy-flattening procedure. Apart from the input-scale adjustment and the choice of output level, it follows [Algorithm 1](#).

[Table 8](#) gives the implemented arithmetic bootstrapping procedure. Unlike [Algorithm 2](#), the implementation switches back to s immediately after the first modulus raising. It then multiplies by $t(X^{N/n})$, retains one copy, and reduces another copy modulo q_0 to form the error branch. This branch switches to s' again before the second modulus raising. The extra key switch is included in the measured running times and output errors. Multiplication by $t(X^{N/n})$ uses an integer plaintext at scale one and consumes no level.

Table 7. Implemented **LazyFlatten** at input level $\ell = \ell_{N \leftarrow n}$ and output level $\tilde{\ell}$. The input scale is S and the output scale is Δ .

Operation	Level	Secret key
$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{N \leftarrow n}^{(S)}), \cdot)$	$\ell \rightarrow 0$	s
$\text{KeySwitch}_{s' \leftarrow s}$	0	$s \rightarrow s'$
$\text{ModRaise}_{\tilde{\ell} + \ell_{n \leftarrow N} \leftarrow 0}$	$0 \rightarrow \tilde{\ell} + \ell_{n \leftarrow N}$	s'
$\text{KeySwitch}_{s \leftarrow s'}$	$\tilde{\ell} + \ell_{n \leftarrow N}$	$s' \rightarrow s$
$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{n \leftarrow N}), \cdot)$	$\tilde{\ell} + \ell_{n \leftarrow N} \rightarrow \tilde{\ell}$	s

Table 8. Implemented **Boot** at input level $\ell = \ell_{N \leftarrow n}$ and output level $\tilde{\ell} \leq \hat{\ell}$. The input scale is S and the output scale is Δ . Each modulus raising is surrounded by switches from s to s' and back to s .

Operation	Level	Secret key
$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{V}_{N \leftarrow n}^{(S)}), \cdot)$	$\ell \rightarrow 0$	s
$\text{ModRaise}_{\tilde{\ell} + \ell_{n \leftarrow N} \leftarrow 0}$ with key switching	$0 \rightarrow \tilde{\ell} + \ell_{n \leftarrow N}$	$s \rightarrow s' \rightarrow s$
$t(X^{N/n}) \cdot \mathbf{ct}$	$\tilde{\ell} + \ell_{n \leftarrow N}$	s
Copy and reduce: $\mathbf{ct}' \leftarrow [\mathbf{ct}]_{q_0}$	$\tilde{\ell} + \ell_{n \leftarrow N} \rightarrow 0$	s
Error branch: $\text{ModRaise}_{L \leftarrow 0}$ with key switching	$0 \rightarrow L$	$s \rightarrow s' \rightarrow s$
$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{2 \leftarrow N}), \mathbf{ct}')$	$L \rightarrow L - \ell_{2 \leftarrow N}$	s
$\text{EvalMod}(\mathbf{ct}')$	$L - \ell_{2 \leftarrow N} \rightarrow \hat{\ell} + \ell_{n \leftarrow 2}$	s
$\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{U}_{n \leftarrow 2}), \mathbf{ct}')$	$\hat{\ell} + \ell_{n \leftarrow 2} \rightarrow \hat{\ell}$	s
Retained branch: $\text{LinTrans}(\text{Ecd}_{\mathbb{C}}(\mathbf{W}_{n \leftarrow N}), \mathbf{ct})$	$\tilde{\ell} + \ell_{n \leftarrow N} \rightarrow \tilde{\ell}$	s
Reduce and subtract: $[\mathbf{ct} - \mathbf{ct}']_{q_{\tilde{\ell}}}$	$\tilde{\ell}$	s

F.3 Implementation of EvalMod

We use Lattigo’s `mod1` circuit, applied separately to the real and imaginary components of the slots. The benchmark parameters use a degree-30 discrete cosine approximation followed by three double-angle steps, with range parameter $K = 16$. The circuit’s input and output normalization factors are folded into $\mathbf{W}_{2 \leftarrow N}$ and $\mathbf{U}_{n \leftarrow 2}$, respectively. The tables use the names of the resulting conceptual transformations from Section 7. Before the coefficient-to-slot transformation, the error branch’s scale metadata is set to Δ so that EvalMod interprets its input with the intended normalization.

The discrete approximation covers inputs near integers of absolute value up to about $K/2$. We choose its range according to the observed overflow, whose magnitude is about 7. This is smaller than the domain extending to $h'/2 + 1 + \varepsilon_{\text{em}}$ assumed in Section 7. A tighter high-probability overflow bound, as discussed in Section 6, would be needed to derive a failure-probability guarantee for these experimental parameters.

F.4 Effect of the RLWE Dimension on Precision

In Table 9, we vary N with $t = X - 2$ and a fixed modulus chain, scale, ephemeral-secret weight, and EvalMod parameters. It compares four lanes ($N/n = 4$) with a fixed lane degree ($n = 2^8$); their common configuration at $N = 2^{10}$ is listed once. In the four-lane sweep, doubling N increases the logarithm of the normalized error by about 1.1 bits. At $N = 2^{11}, \dots, 2^{14}$, the logarithms of the errors for the two choices of n differ by at most 0.3 bits.

Table 9. Normalized output error after Boot. Configurations with $N < 2^{16}$ are insecure with the fixed modulus chain.

RLWE dimension N	Lane degree n	Number of lanes N/n	Normalized output error $\ \tilde{\mathbf{e}}\ _{\infty}/\Delta$
2^{10}	2^8	4	$2^{-41.7}$
2^{11}	2^9	4	$2^{-40.5}$
2^{11}	2^8	8	$2^{-40.3}$
2^{12}	2^{10}	4	$2^{-39.5}$
2^{12}	2^8	16	$2^{-39.4}$
2^{13}	2^{11}	4	$2^{-38.2}$
2^{13}	2^8	32	$2^{-38.2}$
2^{14}	2^{12}	4	$2^{-37.4}$
2^{14}	2^8	64	$2^{-37.1}$
2^{15}	2^{13}	4	$2^{-36.3}$
2^{16}	2^{14}	4	$2^{-35.0}$